



Học viện
Công nghệ Bưu chính Viễn thông

PHÂN TÍCH DỮ LIỆU CHUỖI THỜI GIAN

Học máy và Học sâu

Giới thiệu



- ❑ Trong chương trước, chúng ta đã xây dựng một lý thuyết cơ bản về động lực của chuỗi thời gian và các thống kê mô tả nhiều và độ không chắc chắn trong hành vi của nó. Sau đó, chúng ta sử dụng động lực giả định của quá trình để đưa ra dự đoán.
- ❑ Bây giờ, chúng ta chuyển sang các phương pháp luận mà trong đó chúng ta không đặt ra một quy trình cơ bản hoặc bất kỳ quy tắc nào về quy trình cơ bản đó. Thay vào đó, chúng ta tập trung vào việc xác định các mô hình mô tả hành vi của quy trình theo những cách liên quan đến việc dự đoán kết quả quan tâm
 - Phân loại với dữ liệu chuỗi thời gian
 - Phân cụm với dữ liệu chuỗi thời gian

Giới thiệu



- ❑ Chuỗi thời gian khác dữ liệu thông thường ở 3 điểm:
 - **Phụ thuộc theo thời gian (temporal dependence):** giá trị tại t liên quan trực tiếp tới $t-k$.
 - **Không i.i.d** (independent and identically distributed): nhiều thuật toán ML giả định độc lập sẽ không áp dụng trực tiếp.
 - **Trật tự (order)** cực kỳ quan trọng: shuffle dữ liệu = sai hoàn toàn.
- ❑ Vì vậy các phương pháp học máy cho phân tích chuỗi thời gian phải giải quyết:
 - Sự phụ thuộc của mô hình: dùng các “window” hoặc mô hình sequence.
 - Dự đoán trước một bước hoặc nhiều bước.
 - Các đặc trưng mang đặc thù thời gian.

Giới thiệu



□ Một số lưu ý:

- Trong trường hợp phân cụm và phân loại dựa trên khoảng cách, chúng ta sẽ thấy rằng chúng ta có lựa chọn sử dụng các đặc trưng hoặc sử dụng chuỗi thời gian gốc làm đầu vào. Để sử dụng chính chuỗi thời gian làm đầu vào, chúng ta nghiên cứu một phép đo khoảng cách được gọi là độ xoắn thời gian động, có thể được áp dụng trực tiếp vào chuỗi thời gian, bảo toàn toàn bộ tập thông tin theo thứ tự thời gian trong dữ liệu của chúng ta thay vì thu gọn nó thành một tập hợp các đặc trưng giới hạn.
- Khi chọn các đặc trưng để tạo chuỗi thời gian, hãy đảm bảo chọn các đặc trưng mang tính ergodic, nghĩa là các giá trị được đo sẽ hội tụ về một giá trị ổn định khi thu thập thêm dữ liệu từ cùng một quy trình.

Nội dung



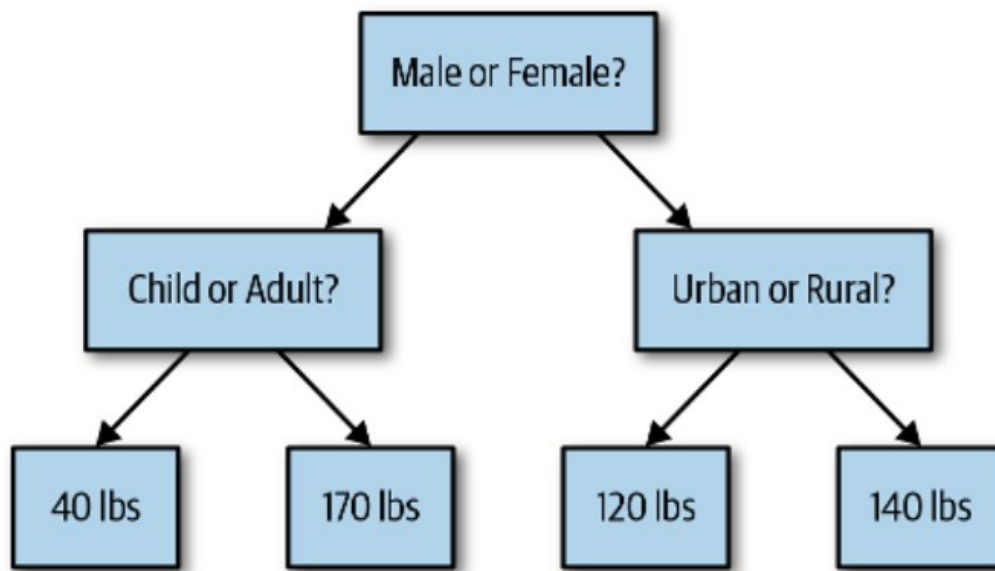
- ❖ Phân loại với dữ liệu chuỗi thời gian
- ❖ Phân cụm với dữ liệu chuỗi thời gian
- ❖ Mạng nơ ron
 - Mạng chuyển tiếp (feed forward network)
 - Cơ chế Attention tận dụng đặc tính thời gian
- ❖ Mạng tích chập (CNN)
- ❖ Mạng hồi qui (RNN)
- ❖ Các chỉ số đánh giá



Phân loại chuỗi thời gian

➤ Phương pháp Cây Quyết định (Decision Tree Methods)

- Chúng phản ánh cách chúng ta đưa ra những quyết định phức tạp: từng bước một và theo một cách phi tuyến tính cao, suy nghĩ về cách một biến số sẽ ảnh hưởng đến quyết định của chúng ta, và sau đó là một biến số khác, giống như một sơ đồ dòng chảy.





Phân loại chuỗi thời gian

➤ Phương pháp Cây Quyết định (Decision Tree Methods)

- Ví dụ: một nhà giao dịch chứng khoán sử dụng các chỉ báo kỹ thuật theo kiểu phân cấp tuần tự, giống như một cây. Trước tiên xem động lượng đang đi theo hướng nào, sau đó kiểm tra mức độ biến động đang diễn biến như thế nào theo thời gian. Câu trả lời cho câu hỏi thứ hai này tương tác với câu hỏi đầu tiên theo kiểu cây phi tuyến tính.
- Ví dụ: một chuyên gia y tế đọc điện não đồ (EEG) hoặc điện tâm đồ (ECG), việc đầu tiên thường là tìm kiếm sự hiện diện của một đặc điểm trước khi xem xét một vài đặc điểm khác, và sau đó lần lượt xem xét một loạt các yếu tố.



Phân loại chuỗi thời gian

➤ Phương pháp Cây Quyết định (Decision Tree Methods)

- Ví dụ: Dự đoán giá trị số (liên tục) trong tương lai.
 - Khi đã biết $X_t = [y_t, y_{t-1}, \dots, y_{t-p}]$.
 - Đầu ra (output): Một giá trị số thực $\hat{y}_{t+1} \in \mathbb{R}$ hoặc vector nhiều bước $[\hat{y}_{t+1}, \hat{y}_{t+2}, \dots, \hat{y}_{t+h}]$
 - Decision Tree học bằng cách chia miền giá trị dựa trên các ngưỡng để dự đoán output. Ví dụ như:
 - Nếu $y_t > 20$ và $y_{t-1} < 5$ thì dự báo sẽ cao.
 - Nếu y_{t-3} rất lớn thì forecast giảm.



Phân loại chuỗi thời gian

➤ Rừng ngẫu nhiên

- Rừng ngẫu nhiên là một mô hình mà chúng ta sử dụng không chỉ một mà là nhiều cây quyết định. Kết quả cuối cùng dựa trên việc tính trung bình các kết quả đầu ra của những cây này.
- Rừng ngẫu nhiên được xây dựng theo các tham số xác định số lượng cây, cũng như độ sâu tối đa cho phép của các cây đó.
 - Đối với mỗi cây riêng lẻ, một mẫu ngẫu nhiên của dữ liệu và các đặc trưng của nó được sử dụng để huấn luyện cây đó.
 - Các cây thường được tham số hóa khá đơn giản để có thể tránh được hiện tượng quá khớp (overfitting). Không có mô hình nào đặc biệt tốt nhưng tất cả đều đủ tổng quát để tránh "bẫy" trong dữ liệu.



Phân loại chuỗi thời gian

➤ Rừng ngẫu nhiên

- Rừng ngẫu nhiên không phải là một công cụ tốt để làm việc với dữ liệu chuỗi thời gian ở dạng thô. Chúng có thể hữu ích khi làm việc với dữ liệu này sau khi được nén thành các đặc trưng tóm tắt.
 - Cô đọng các chuỗi thời gian cực dài thành một số ít đặc trưng và tìm ra một mô hình với độ chính xác hợp lý và tăng hiệu quả tính toán.
 - Rừng ngẫu nhiên giúp giảm thiểu rủi ro quá khớp, đây là một vấn đề đối với phân tích chuỗi thời gian do sự tương tác không mong muốn giữa quá khớp và vấn đề nhìn trước.
- Rừng ngẫu nhiên được triển khai cho phân loại chuỗi thời gian thành công hơn so với dự báo chuỗi thời gian.



Phân loại chuỗi thời gian

- **Cây tăng cường gradient** đã trở thành phương pháp tăng cường phổ biến, và đặc biệt thành công đối với chuỗi thời gian.
- Tăng cường là một cách khác để xây dựng một tập hợp các yếu tố dự báo.
 - Tăng cường tạo ra các mô hình tuần tự với ý tưởng rằng các mô hình sau sẽ sửa các lỗi của các mô hình trước đó và dữ liệu không khớp của các mô hình trước đó sẽ được các mô hình sau này đánh giá trọng số cao hơn.
 - Các cây tăng cường gradient cho dự đoán thường vượt trội hơn các mô hình thống kê truyền thống khi được cung cấp bộ dữ liệu đủ lớn. Chúng tiếp cận mô hình "tự động" trong khả năng loại bỏ các đặc điểm không liên quan hoặc nhiễu và tập trung vào các đặc điểm quan trọng nhất.



Phân loại chuỗi thời gian

➤ Cây tăng cường gradient

Gradient Boosting = xây các cây liên tiếp, mỗi cây học để “sửa lỗi” của cây trước.

- Công thức ý tưởng: $f_{t+1}(x) = f_t(x) + \eta \cdot h_t(x)$

Trong đó:

- h_t : cây học phần residual
- η : learning rate
- f_t : mô hình sau bước t

→ Tạo ra mô hình cực mạnh cho phi tuyến phức tạp.

Gradient Boosting (XGBoost, LightGBM, CatBoost)



Phân loại chuỗi thời gian

➤ Cây tăng cường gradient

- **XGBoost** hoạt động bằng cách xây dựng các cây theo trình tự, với mỗi cây tìm cách dự đoán phần dư của tổ hợp các cây trước đó.
 - Cây đầu tiên được XGBoost xây dựng sẽ cố gắng khớp trực tiếp dữ liệu (một danh mục hoặc một giá trị số).
 - Cây thứ hai sẽ cố gắng dự đoán giá trị thực trừ đi giá trị dự đoán.
 - Cây thứ ba sẽ cố gắng dự đoán giá trị thực trừ đi giá trị dự đoán của cây đầu tiên trừ đi dự đoán của cây thứ hai về phần dư của cây đầu tiên.



Phân loại chuỗi thời gian

➤ Cây tăng cường gradient

- **XGBoost** không chỉ đơn giản xây dựng các mô hình vô hạn, cố gắng giảm thiểu phần dư của phần dư dự đoán của phần dư dự đoán vô hạn.
 - Thuật toán XGBoost giảm thiểu một hàm mất mát cũng bao gồm một số hạng phạt cho độ phức tạp của mô hình, và số hạng phạt này giới hạn số lượng cây sẽ được tạo ra.
 - Cũng có thể trực tiếp giới hạn số lượng cây được tạo ra.



Phân loại chuỗi thời gian

- **Mục tiêu phân loại trong phân tích chuỗi thời gian (Time Series Classification):** Gán nhãn phân loại cho mỗi điểm hoặc cho toàn chuỗi. Không dự đoán giá trị, mà dự đoán loại của sự kiện/chuỗi.
- **Đầu ra (output):** Một nhãn rời rạc $\hat{y}_t \in \{0,1\}$ hoặc $\{A, B, C\}$
 - **Phân loại điểm (point-wise classification):**
 - Phát hiện **anomaly** (bình thường / bất thường).
 - Nhận diện sự kiện: “lên trend”, “xuống trend”, “đột biến”.
 - **Phân loại chuỗi (whole series classification):**
 - Nhận dạng tín hiệu ECG (nhịp tim bình thường / nhịp thất).
 - Phân loại mẫu âm thanh (bước chân, tiếng gõ, tiếng động cơ).
 - Phân loại chuyển động tay trong cảm biến wearable.
 - Nhận dạng hành vi robot (đi thẳng / quay trái / quay phải).



Phân cụm

- **Phân cụm cho chuỗi thời gian** có thể được sử dụng cho cả phân loại và dự báo.
- Ý tưởng chung của phân cụm là các điểm dữ liệu tương tự nhau tạo thành các nhóm có ý nghĩa cho mục đích phân tích.
 - Trong trường hợp phân loại, chúng ta có thể sử dụng các thuật toán phân cụm để xác định số lượng cụm mong muốn trong giai đoạn huấn luyện. Sau đó, chúng ta có thể sử dụng các cụm này để thiết lập các loại chuỗi thời gian và nhận biết khi nào các mẫu mới thuộc về một nhóm cụ thể.
 - Trong trường hợp dự báo, ứng dụng có thể là phân cụm thuần túy hoặc có thể được lấy cảm hứng từ phân cụm dưới dạng sử dụng các số liệu khoảng cách liên quan.



Phân cụm

- **Phân cụm cho chuỗi thời gian** có thể được sử dụng cho cả phân loại và dự báo.
 - Giả sử ta chỉ quan sát N bước đầu tiên của nó, từ đó chúng ta muốn dự báo giá trị của nó tại bước thời gian $N + h$. Trong trường hợp như vậy, có một vài tùy chọn.
 - Sử dụng lớp thành viên để tạo dự báo dựa trên hành vi điển hình trong lớp đó.
 - Trước tiên hãy xác định cụm mà mẫu chuỗi thời gian thuộc về dựa trên N bước thời gian đầu tiên của nó, sau đó suy ra hành vi có thể xảy ra trong tương lai dựa trên thành viên cụm.
 - Lưu ý rằng bạn nên thực hiện phân cụm ban đầu cho tất cả các chuỗi thời gian dựa trên N bước đầu tiên của chúng, thay vì tất cả các phần của chuỗi thời gian để tránh việc nhìn trước.



Phân cụm

➤ **Phân cụm cho chuỗi thời gian** có thể được sử dụng cho cả phân loại và dự báo.

- Giả sử ta chỉ quan sát N bước đầu tiên của nó, từ đó chúng ta muốn dự báo giá trị của nó tại bước thời gian $N + h$. Trong trường hợp như vậy, có một vài tùy chọn.
 - Dự đoán hành vi tương lai của một chuỗi thời gian mẫu dựa trên hành vi của các lân cận gần nhất của nó trong không gian mẫu.
 - Dựa trên các số liệu từ N bước thời gian đầu tiên, hãy tìm (các) lân cận gần nhất của một mẫu chuỗi thời gian mà quỹ đạo đầy đủ đã biết.
 - Sau đó, tính trung bình hành vi $N + h$ của các lân cận gần nhất này, và đây là dự báo của bạn cho mẫu hiện tại.



Phân cụm

- Khi chạy phân tích cụm, chúng ta phải chọn một **chỉ số khoảng cách**.
 - Trong cả phân loại và dự báo, điều quan trọng nhất cần cân nhắc là cách đánh giá tính tương đồng giữa các chuỗi thời gian.
 - Chúng ta có hai loại tùy chọn đo lường khoảng cách rộng khi áp dụng các kỹ thuật phân cụm vào dữ liệu chuỗi thời gian:
 - Khoảng cách dựa trên các đặc điểm: Tạo các đặc điểm cho chuỗi thời gian và coi chúng là tọa độ để tính toán dữ liệu.
 - Khoảng cách dựa trên dữ liệu chuỗi thời gian thô: Tìm cách xác định mức độ "gần nhau" của các chuỗi thời gian khác nhau.

Phân cụm

- Khi chạy phân tích cụm, chúng ta phải chọn một **chỉ số khoảng cách**.
- Độ xoắn thời gian động (Dynamic time warping - **DTW**) là một trong những kỹ thuật **so sánh chuỗi thời gian mạnh nhất**, đặc biệt khi hai chuỗi **không đồng tốc, không thẳng hàng**, hoặc có **tốc độ thay đổi khác nhau**. Các quy tắc của DTW như sau:
 - Mỗi điểm trong một chuỗi thời gian phải được khớp với ít nhất một điểm của chuỗi thời gian khác.
 - Chỉ số đầu tiên và cuối cùng của mỗi chuỗi thời gian phải được khớp với các phần tương ứng của chúng trong chuỗi thời gian khác.
 - Việc ánh xạ các điểm phải sao cho thời gian di chuyển về phía trước chứ không phải ngược lại.

Phân cụm

➤ Khi chạy phân tích cụm, chúng ta phải chọn một **chỉ số khoảng cách**.

- Độ xoắn thời gian động (Dynamic time warping - **DTW**)

Ví dụ 2 chuỗi:

Chuỗi A (đi nhanh): 1 2 3 4 5

Chuỗi B (đi chậm): 1 1.5 2 3 4 5

- Nếu so bằng Euclid: Các điểm không khớp thời gian → khoảng cách lớn, sai.
- **DTW cho phép bẻ cong trục thời gian để “căn thẳng” hai chuỗi lại.**
 - Cho phép khớp A[2] với B[2] **hoặc** B[1]
 - Khớp A[3] với B[3] **hoặc** B[2]
 - Khớp A[4] với B[4] **hoặc** B[3]
 - ...



Phân cụm

➤ Khi chạy phân tích cụm, chúng ta phải chọn một **chỉ số khoảng cách**.

- Công thức **DTW** theo quy hoạch động

- Cho hai chuỗi:

$$A = (a_1, a_2, \dots, a_n)$$

$$B = (b_1, b_2, \dots, b_m)$$

- Tạo ma trận chi phí $n \times m$: mỗi vị trí (i, j) chứa khoảng cách của hai điểm tại thời điểm đó và đường đi tối ưu đến đó.

$$D(i, j) = |a_i - b_j| + \min(D(i-1, j), D(i, j-1), D(i-1, j-1))$$

- Cuối cùng: $DTW(A, B) = D(n, m)$



Phân cụm

- Khi chạy phân tích cụm, chúng ta phải chọn một **chỉ số khoảng cách**.
 - Các thước đo khác:
 - Khoảng cách Fréchet: bề cong thời gian của các đường cong để giảm thiểu khoảng cách giữa hai đường cong sau đó đo khoảng cách.
 - Hệ số tương quan Pearson: phép đo tương quan giữa hai chuỗi. Phương pháp này tương đối dễ tính toán nhưng yêu cầu các chuỗi thời gian phải có cùng số điểm dữ liệu.
 - Dây con chung dài nhất: để xem xét tính tương đồng của hai chuỗi thời gian, chúng ta có thể xác định độ dài của dây các giá trị liên tiếp giống hệt nhau. Lưu ý: vị trí chính xác của chúng trong chuỗi thời gian không bắt buộc phải khớp.



Mạng nơ-ron

- **Học sâu** cho chuỗi thời gian là một hướng nghiên cứu tương đối mới, nhưng đầy hứa hẹn.
- Đây là một kỹ thuật rất linh hoạt, mang lại lợi thế cho việc phân tích chuỗi thời gian. Triển vọng nhất là khả năng mô hình hóa hành vi thời gian cực kỳ phức tạp và phi tuyến tính mà không cần phải ước lượng các dạng hàm như các phương pháp thống kê.
 - Không yêu cầu tính dừng. Mặc dù không có yêu cầu về tính dừng đối với học sâu áp dụng cho chuỗi thời gian, nhưng trên thực tế, học sâu không thực hiện tốt việc khớp dữ liệu với xu hướng trừ khi các kiến trúc chuẩn được điều chỉnh để phù hợp với xu hướng. Vì vậy, chúng ta vẫn cần xử lý trước dữ liệu hoặc kỹ thuật của mình.
 - Học sâu rất linh hoạt về mô hình và bản chất của các đầu vào. Dữ liệu không đồng nhất có thể là một thách thức đối với nhiều kỹ thuật học máy thường được áp dụng, trong khi nó lại khá phổ biến đối với các mô hình học sâu.

Mạng nơ-ron

➤ **Học sâu** mô tả một nhánh của học máy, trong đó một "đồ thị" được xây dựng để kết nối các nút đầu vào với một cấu trúc phức tạp gồm các nút và cạnh.

- Giả sử bạn có chuỗi: temps = [30, 31, 29, 32, 33, 34, 32, 31, 30, 29,...]
- Window size = 7 → sinh ra:

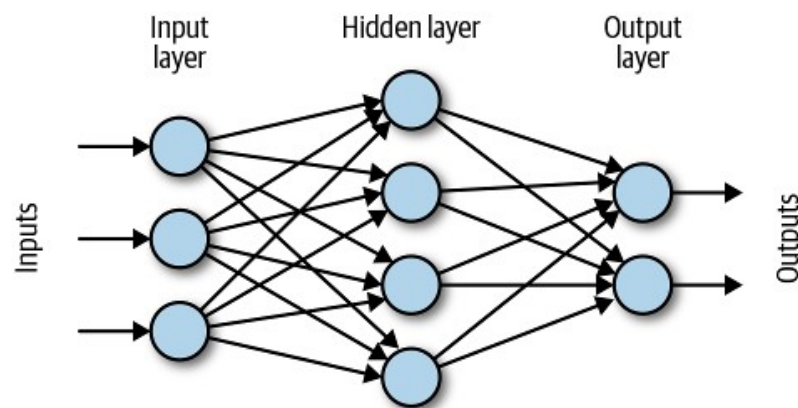
$X_{train} = [[30, 31, 29, 32, 33, 34, 32],$
[31, 29, 32, 33, 34, 32, 31],
[29, 32, 33, 34, 32, 31, 30],
[32, 33, 34, 32, 31, 30, 29],
....]

$y_{train} = [31, 30, 29, 28, ...]$

- Huấn luyện: `model.fit(X_train, y_train, ...)`
- Dự đoán: Ta lấy 7 ngày gần nhất và cho vào model:

$input_last_7 = [31, 32, 33, 34, 30, 29, 30]$

$predict = model.predict(input_last_7)$

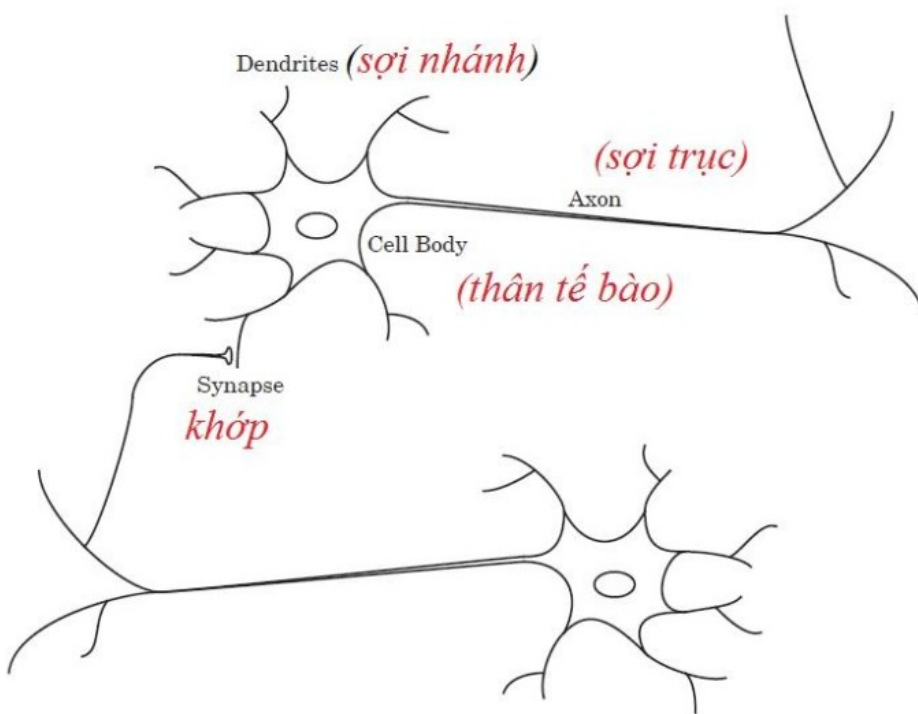




Mạng nơ-ron

➤ Mô hình nơ-ron

- **Nơ-ron sinh học:** Sợi nhánh, sợi trục và khớp
 - Não: 10^{11} nơ – ron
 - Nơ-ron: 10^4 liên kết
 - Chức năng của mạng nơ-ron được hình thành bởi sự sắp xếp các nơ-ron và độ lớn của các khớp thần kinh, gây ra bởi một quá trình hóa học phức tạp.



Mạng nơ-ron sinh học



Mạng nơ-ron

➤ Mô hình nơ-ron

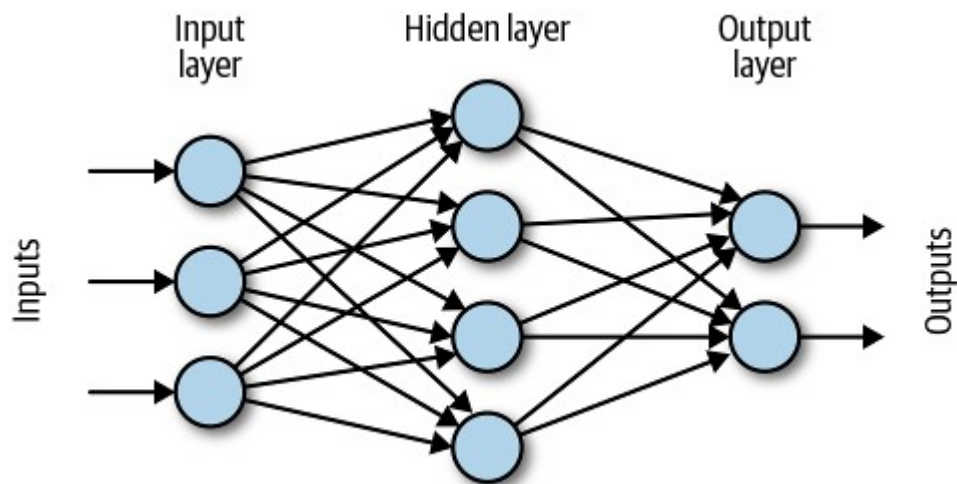
- **Nơ-ron sinh học:**

- Chức năng của mạng nơ-ron được hình thành bởi sự sắp xếp các nơ-ron và độ lớn của các khớp thần kinh, gây ra bởi một quá trình hóa học phức tạp.
 - Một phần của bộ não được hình thành khi sinh ra.
 - Những phần khác sẽ phát triển thông qua học, do có những liên kết mới được hình thành và các liên cũ mất đi.
 - Cấu trúc mạng nơ-ron tiếp tục thay đổi theo thời gian trong đời. Những thay đổi sau này có xu hướng chủ yếu làm mạnh hoặc yếu đi các khớp thần kinh

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Mạng nơ-ron nhân tạo** mô phỏng hoạt động của mạng nơ-ron sinh học và:
 - Các khối tạo ra mạng nơ-ron là những thiết bị tính toán đơn giản.
 - Các liên kết giữa các nơ-ron tạo ra chức năng của mạng.



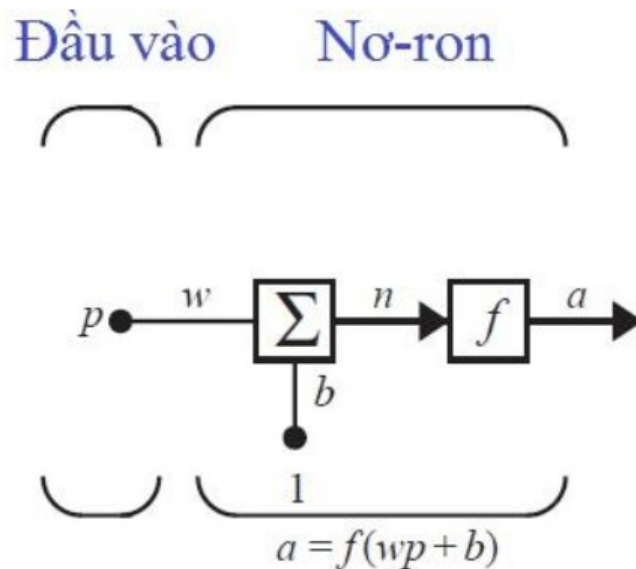
Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo

Nơ ron một đầu vào

- ☐ p – đầu vào của nơ-ron
- ☐ w – trọng số
- ☐ b – ngưỡng (bias)
- ☐ Σ – bộ tổng
- ☐ n – đầu vào net (net input)
- ☐ a – đầu ra của nơ-ron
- ☐ f – hàm truyền



$$n = wp + b$$

$$a = f(n) = 1$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo

Hàm truyền còn gọi là hàm kích hoạt

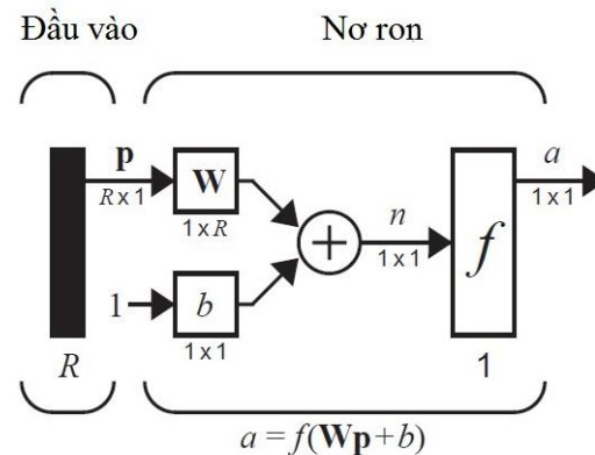
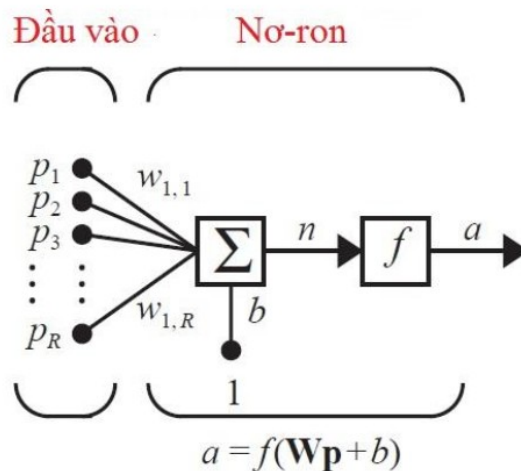
Tên hàm	Mô tả toán học	Ứng dụng
$a = \text{hardlim}(n)$	$a = \begin{cases} 0, & \text{nếu } n < 0 \\ 1, & \text{nếu } 0 \leq n \end{cases}$	Mạng perceptron
$a = \text{purelin}(n)$	$a = n$	Mạng Adaline
$a = \text{logsig}(n)$	$a = \frac{1}{1 + e^{-n}}$	Mạng nhiều lớp, thuật toán lan truyền ngược
$a = \text{tansig}(n)$	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$	
$a = \text{poslin}(n)$	$a = \begin{cases} 0, & \text{nếu } n < 0 \\ n, & \text{nếu } 0 \leq n \end{cases}$	Mạng Hamming
$a = \text{satlins}(n)$	$a = \begin{cases} -1, & n < -1 \\ n, & -1 \leq n \leq 1 \\ 1, & n > 1 \end{cases}$	Mạng Hopfield

Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo
- Nơ-ron nhiều đầu vào

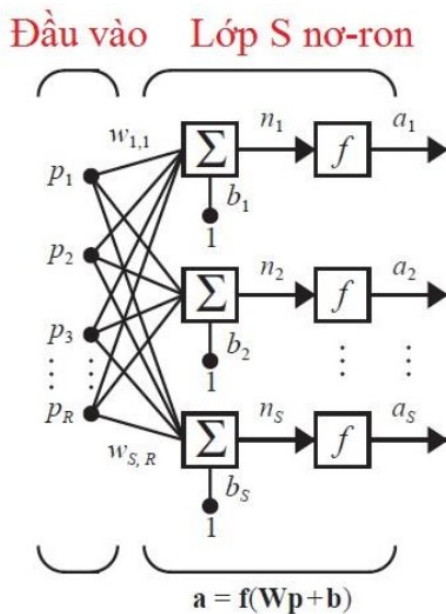
$$\bullet \mathbf{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_R \end{bmatrix}; \mathbf{W} = \begin{bmatrix} w_{1,1} \\ w_{1,2} \\ \vdots \\ w_{1,R} \end{bmatrix}^T; n = b + \sum_{i=1}^R w_{1,i} p_i = \mathbf{W}\mathbf{p} + b$$



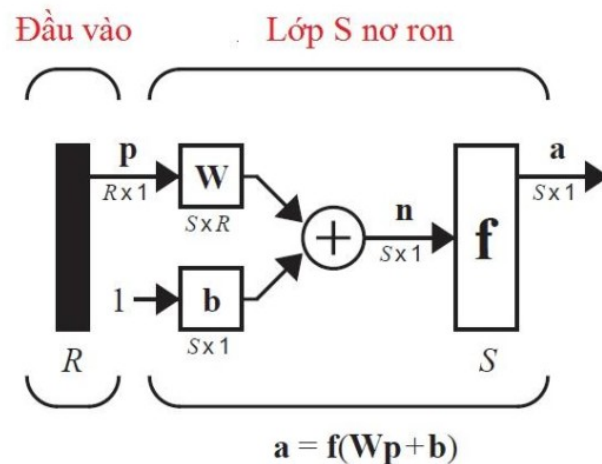
Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo
- Lớp nơ-ron



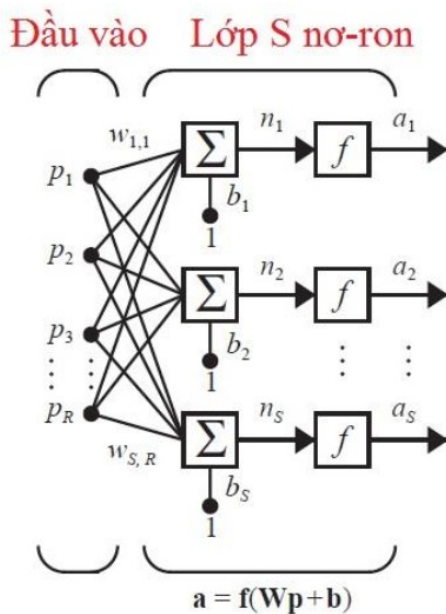
$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,R} \\ w_{2,1} & w_{2,2} & \dots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \dots & w_{S,R} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_S \end{bmatrix}$$



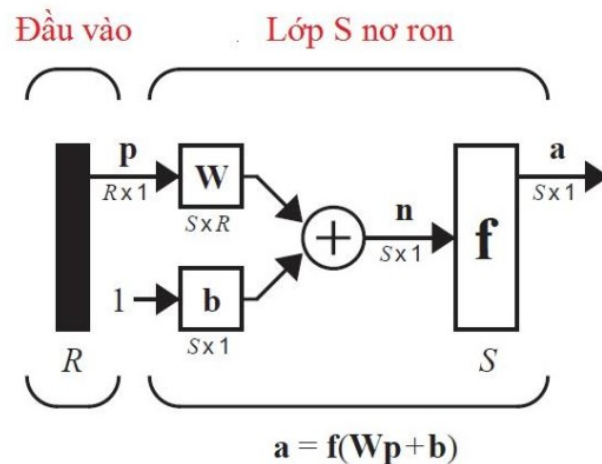
Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo
- Lớp nơ-ron



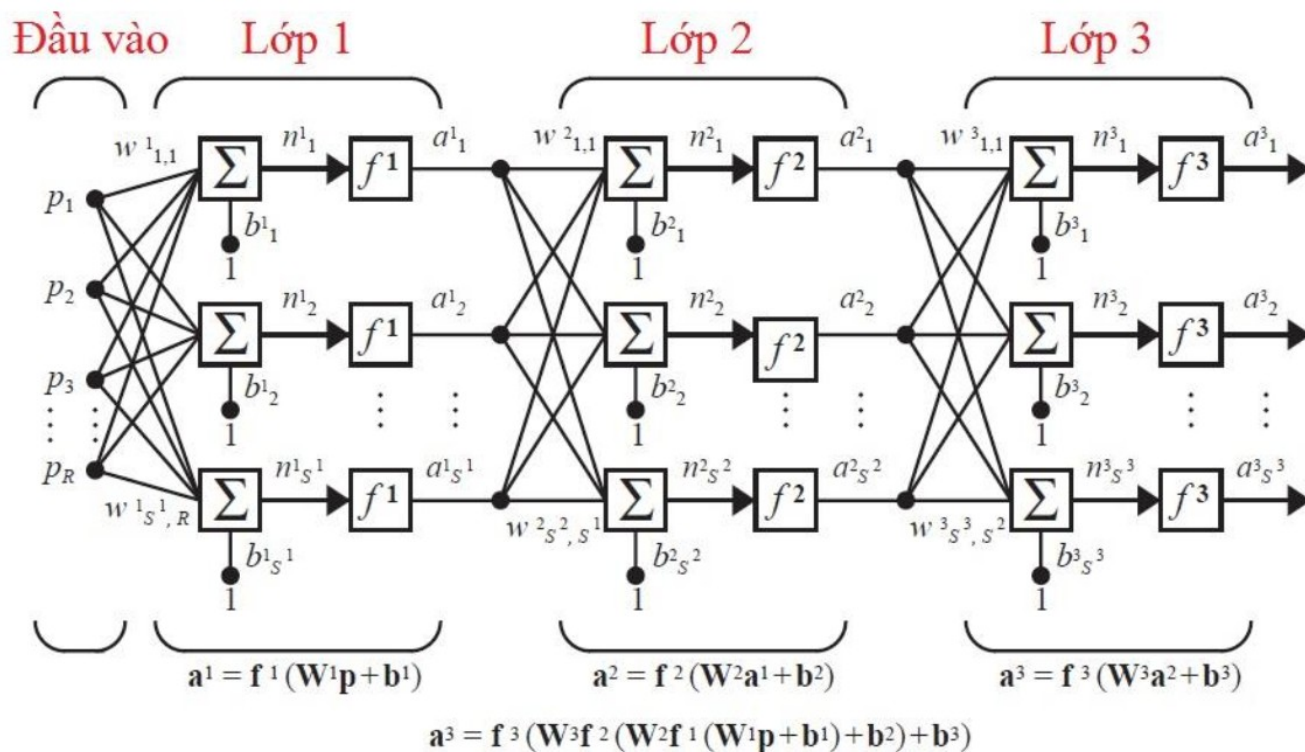
$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,R} \\ w_{2,1} & w_{2,2} & \dots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \dots & w_{S,R} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_S \end{bmatrix}$$



Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng nơ-ron nhân tạo
- Mạng nhiều lớp





Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training):** Là một thủ tục dùng để thay đổi các trọng số và bias nhằm mục đích huấn luyện mạng để thực hiện một số nhiệm vụ.
- **Có 3 loại luật học:**
 - Học có giám sát
 - Học không có giám sát
 - Học gia cố
- **Học có giám sát:**
 - Mẫu huấn luyện $\{\mathbf{p}_1; \mathbf{t}_1\}$, $\{\mathbf{p}_2; \mathbf{t}_2\}$, ... , $\{\mathbf{p}_Q; \mathbf{t}_Q\}$
 - Điều chỉnh các trọng số và bias sao cho đầu ra của mạng nơ-ron bám lấy đầu ra mẫu.



Mạng nơ-ron

➤ Mô hình nơ-ron

- Huấn luyện (Training):

- Xét mạng có M lớp, R đầu vào và S^M đầu ra.
- Tập mẫu: $\Omega = \{\{\mathbf{p}_i; \mathbf{t}_i\} : i = 1, \dots, Q\}$, $\mathbf{p}_i$ - đầu vào mẫu và $\mathbf{t}_i$ - đầu ra mẫu (đích).
- Hàm mục tiêu:

$$J = \frac{1}{Q} \sum_{q=1}^Q J_q \rightarrow \min$$

$$J_q = \mathbf{e}_q^T \mathbf{e}_q$$

$$\mathbf{e}_q = \mathbf{t}_q - \mathbf{a}_q^M$$

$\mathbf{a}_q^M$ - đầu ra tương ứng với đầu vào $\mathbf{p}_q$.



Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training):**
 - Phương pháp gradient:

For $k = 1:K$

For $q \in \text{mini} - \text{batch samples}$

$$w_{i,j}^m(k+1) = w_{i,j}^m(k) - \alpha \frac{\partial J_q}{\partial w_{i,j}^m},$$

$w_{i,j}^m$: trọng số của nơ-ron thứ i thuộc lớp m , liên kết với đầu ra của nơ-ron thứ j thuộc lớp $m-1$;

$\alpha > 0$: tốc độ học.

$$b_i^m(k+1) = b_i^m(k) - \alpha \frac{\partial J_q}{\partial b_i^m},$$

b_i^m : ngưỡng của nơ-ron thứ i thuộc lớp m .

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training):**
 - Tính gradient dựa trên quy tắc chuỗi

$$\frac{\partial J_q}{\partial w_{i,j}^m} = \frac{\partial J_q}{\partial n_i^m} \frac{\partial n_i^m}{\partial w_{i,j}^m}$$

$$n_i^m = b_i^m + \sum_{v=1}^{s^{m-1}} w_{i,v}^m a_v^{m-1},$$

$$\frac{\partial n_i^m}{\partial w_{i,j}^m} = a_j^{m-1}$$

$$\frac{\partial n_i^m}{\partial b_i^m} = 1.$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training):**

- Tính gradient dựa trên quy tắc chuỗi

Ký hiệu:

$$s_i^m = \frac{\partial J_q}{\partial n_i^m}$$

$$\mathbf{s}^m = \frac{\partial J_q}{\partial \mathbf{n}^m} = \left[\frac{\partial J_q}{\partial n_1^m} \quad \frac{\partial J_q}{\partial n_2^m} \quad \dots \quad \frac{\partial J_q}{\partial n_i^m} \right]^T$$

$$\dot{\mathbf{F}}^m(\mathbf{n}^m) = \begin{bmatrix} \dot{f}^m(n_1^m) & 0 & \dots & 0 \\ 0 & \dot{f}^m(n_2^m) & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \dot{f}^m(n_{S_m}^m) \end{bmatrix}$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- Huấn luyện (Training):

- Tính gradient dựa trên quy tắc chuỗi
Các bước tính toán trung gian

$$\begin{aligned}\frac{\partial n_i^{m+1}}{\partial n_j^m} &= \frac{\partial \left(\sum_{l=1}^{S^m} w_{i,l}^{m+1} a_l^m + b_i^{m+1} \right)}{\partial n_j^m} \\ &= w_{i,j}^{m+1} \frac{\partial a_j^m}{\partial n_j^m} \\ &= w_{i,j}^{m+1} \frac{\partial f^m(n_j^m)}{\partial n_j^m} \\ &= w_{i,j}^{m+1} \dot{f}^m(n_j^m).\end{aligned}$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training):**
 - Tính gradient dựa trên quy tắc chuỗi
 - Các bước tính toán trung gian

$$\frac{\partial \mathbf{n}^{m+1}}{\partial \mathbf{n}^m} = \begin{bmatrix} \frac{\partial n_1^{m+1}}{\partial n_1^m} & \frac{\partial n_1^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_1^{m+1}}{\partial n_{S^m}^m} \\ \frac{\partial n_2^{m+1}}{\partial n_1^m} & \frac{\partial n_2^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_2^{m+1}}{\partial n_{S^m}^m} \\ \vdots & \vdots & & \vdots \\ \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_1^m} & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_{S^m}^m} \end{bmatrix} = \mathbf{W}^{m+1} \dot{\mathbf{F}}^m(\mathbf{n}^m)$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- Huấn luyện (Training):

- Tính gradient dựa trên quy tắc chuỗi

$$\mathbf{s}^M \rightarrow \mathbf{s}^{M-1} \rightarrow \dots \rightarrow \mathbf{s}^2 \rightarrow \mathbf{s}^1$$

$$\begin{aligned} s_i^M &= \frac{\partial J_q}{\partial n_i^M} \\ &= \frac{\partial (\mathbf{t}_q - \mathbf{a}_q)^T (\mathbf{t}_q - \mathbf{a}_q)}{\partial n_i^M} \\ &= \frac{\partial \sum_{j=1}^{S^M} (t_j - a_j)^2}{\partial n_i^M} \\ &= -2(t_i - a_i) \frac{\partial a_i}{\partial n_i^M}. \end{aligned}$$



$$\mathbf{s}^M = -2\dot{\mathbf{F}}^M(\mathbf{n}^M)(\mathbf{t}_q - \mathbf{a}_q)$$

$$\begin{aligned} \mathbf{s}^m &= \frac{\partial J_q}{\partial \mathbf{n}^m} \\ &= \left(\frac{\partial \mathbf{n}^{m+1}}{\partial \mathbf{n}^m} \right)^T \frac{\partial J_q}{\partial \mathbf{n}^{m+1}} \\ &= \dot{\mathbf{F}}^m(\mathbf{n}^m)(\mathbf{W}^{m+1})^T \frac{\partial J_q}{\partial \mathbf{n}^{m+1}} \\ &= \dot{\mathbf{F}}^m(\mathbf{n}^m)(\mathbf{W}^{m+1})^T \mathbf{s}^{m+1}. \end{aligned}$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Huấn luyện (Training): tóm tắt**

- Quá trình khởi tạo mạng: Chọn giá trị ban đầu cho các tham số.
- Quá trình lan truyền thuận

$$\mathbf{a}^0 = \mathbf{p},$$

$$\mathbf{a}^{m+1} = \mathbf{f}^{m+1}(\mathbf{W}^{m+1} \mathbf{a}^m + \mathbf{b}^{m+1}), \forall m = 0, 1, \dots, M-1.$$

$$\mathbf{a} = \mathbf{a}^M.$$

- Quá trình lan truyền ngược độ nhạy

$$\mathbf{s}^M = -2\dot{\mathbf{F}}^M(\mathbf{n}^M)(\mathbf{t} - \mathbf{a}),$$

$$\mathbf{s}^m = \dot{\mathbf{F}}^m(\mathbf{n}^m)(\mathbf{W}^{m+1})^T \mathbf{s}^{m+1}, \forall m = M-1, \dots, 2, 1.$$

- Quá trình cập nhật các tham số của mạng

$$\mathbf{W}^m(k+1) = \mathbf{W}^m(k) - \alpha \mathbf{s}^m (\mathbf{a}^{m-1})^T$$

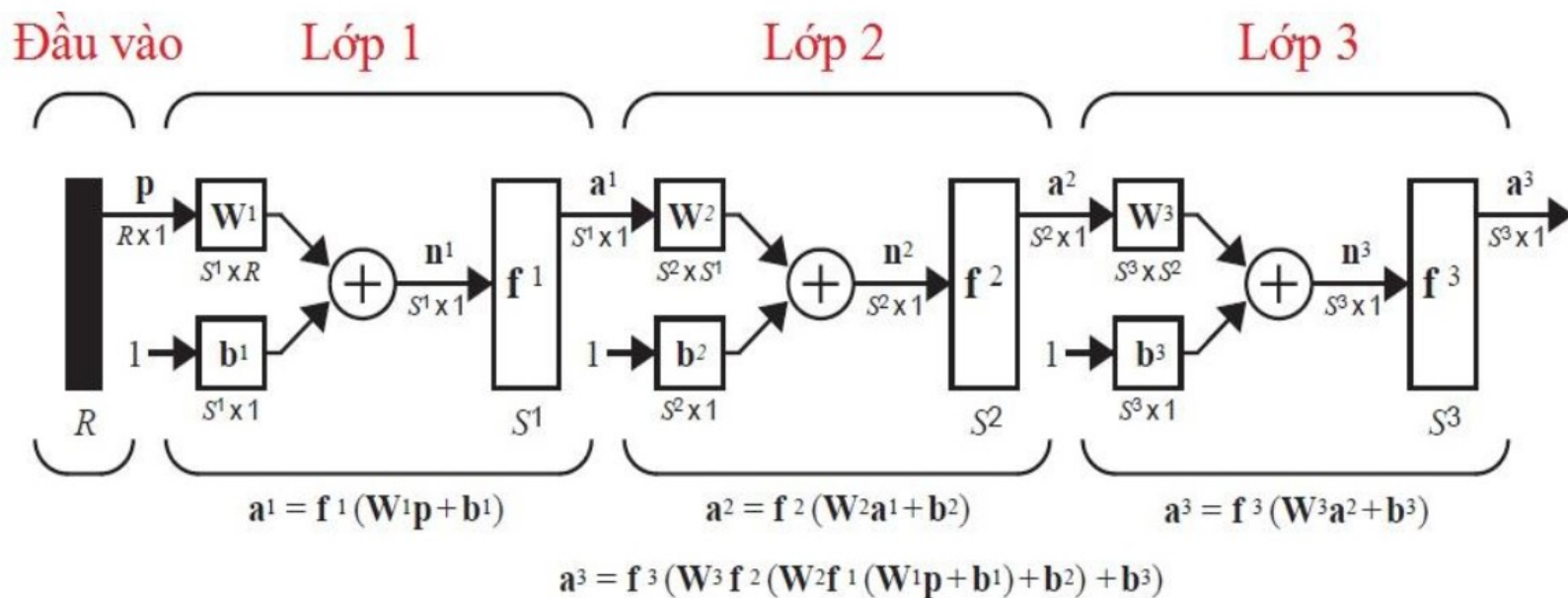
$$\mathbf{b}^m(k+1) = \mathbf{b}^m(k) - \alpha \mathbf{s}^m$$

Mạng nơ-ron

➤ Mô hình nơ-ron

- Mạng chuyển tiếp (Feed Forward Networks)

Mạng nhiều lớp





Mạng nơ-ron

➤ Mô hình nơ-ron

- **Mạng chuyển tiếp (Feed Forward Networks)**

- Mạng truyền thẳng có khả năng song song hóa cao, nghĩa là chúng có hiệu suất khá tốt (tính toán nó rất nhanh).
- Mạng truyền thẳng là một phép thử tốt để xác định xem chuỗi của bạn có thực sự tồn tại động lực theo thời gian phức tạp hay không.
- Các thành phần mạng truyền thẳng thường được tích hợp vào các kiến trúc học sâu chuỗi thời gian lớn hơn và phức tạp hơn.



Mạng nơ-ron

➤ Mô hình nơ-ron

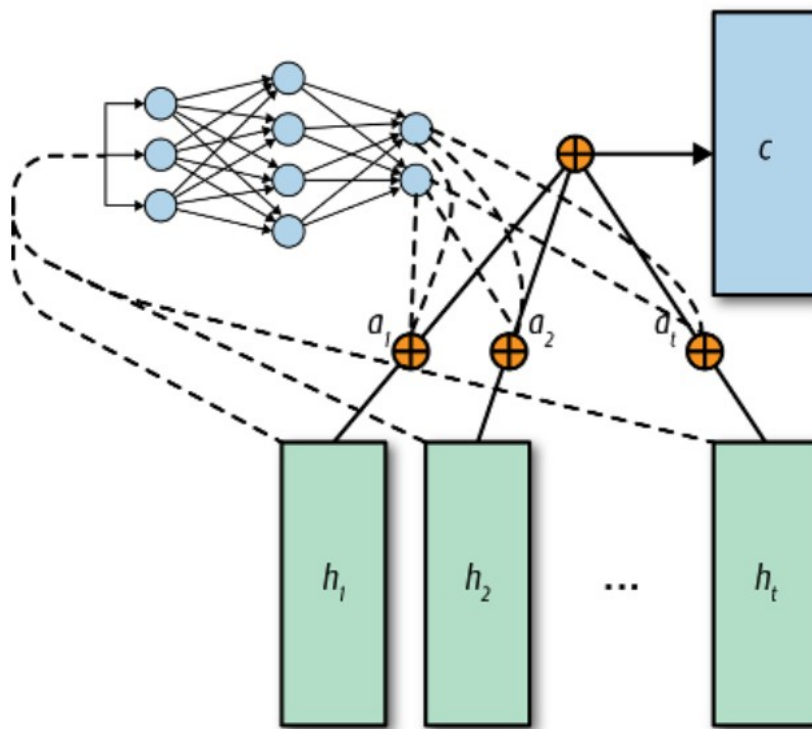
- **Mạng chuyển tiếp (Feed Forward Networks)**

- **Cơ chế Chú ý (Attention Mechanism)** được sử dụng để tạo ra các biến thể kiến trúc cho mạng truyền thẳng sao cho có thể nâng cao hiệu suất của chúng trên dữ liệu tuần tự.
 - Sự chú ý (**Attention**) mô tả các cơ chế được thêm vào mạng nơ-ron cho phép nó học được phần nào của chuỗi cần tập trung vào và phần nào của chuỗi đầu vào có thể liên quan đến đầu ra mong muốn.
 - Ý tưởng về sự chú ý là kiến trúc mạng nơ-ron nên cung cấp một cơ chế để mô hình học được thông tin nào là quan trọng vào thời điểm nào. Điều này được thực hiện thông qua các trọng số chú ý, phù hợp với từng bước thời gian để mô hình học được cách kết hợp thông tin từ các bước thời gian khác nhau.

Mạng nơ-ron

➤ Mô hình nơ-ron

- **Mạng chuyển tiếp (Feed Forward Networks)**
 - Sử dụng **Cơ chế Chú ý (Attention Mechanism)** để giúp mạng truyền thẳng nhận thức thời gian tốt hơn



A “feed forward attention mechanism.”



Mạng nơ-ron tích chập

- ❑ Hầu hết các chức năng đáng kinh ngạc, phá kỷ lục, giống con người mà máy tính đạt được trong những năm gần đây đều là nhờ các kiến trúc tích chập cực kỳ tinh vi và phức tạp.
- ❑ Ý tưởng về tích chập khá trực quan và đã có từ rất lâu trước khi học sâu ra đời. Tích chập từ lâu đã được sử dụng trong các hình thức xử lý hình ảnh và nhận dạng hình ảnh, bắt đầu với một thứ đơn giản như phép làm mờ Gauss.
- ❑ Vì nhiều lý do, tích chập truyền thống không phù hợp với chuỗi thời gian. Nguyên nhân chủ yếu là do một đặc điểm chính của tích chập, tất cả các không gian đều được xử lý như nhau. Điều này có ý nghĩa đối với hình ảnh, nhưng không đúng với chuỗi thời gian, trong đó một số điểm thời gian nhất thiết phải gần nhau hơn những điểm khác.
- ❑ Tích chập thường được sử dụng như một thành phần của mạng để phân tích chuỗi thời gian hơn là toàn bộ mạng. Ngoài ra, điều này dẫn đến việc chúng được nghiên cứu thường xuyên hơn cho mục đích phân loại hơn là dự báo.



Mạng nơ-ron tích chập

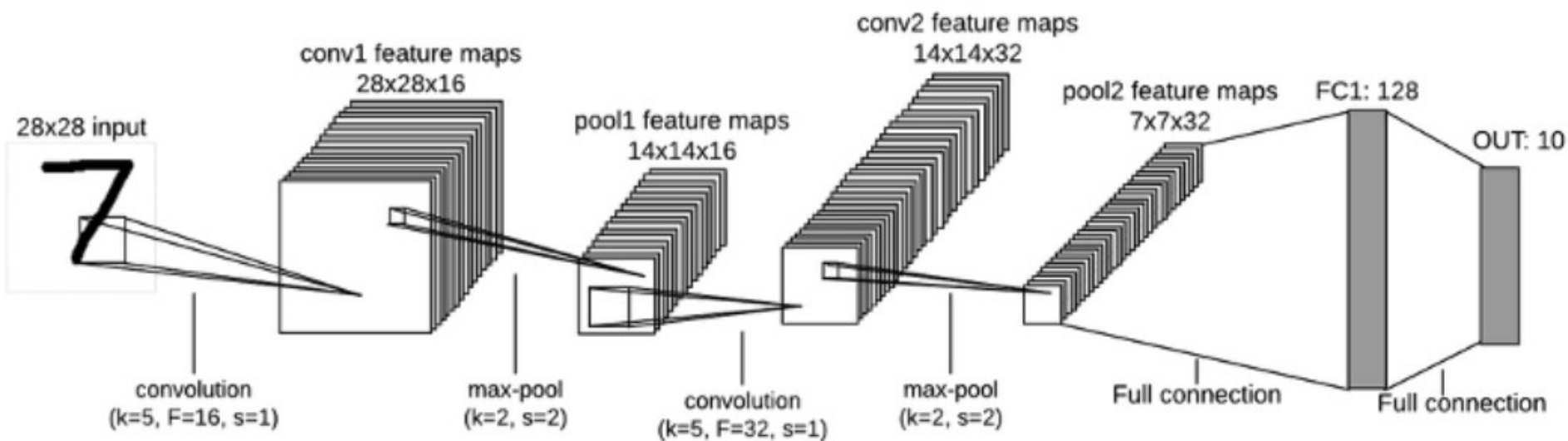
➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

Cấu trúc bao gồm:

- Các tầng tích chập
 - Lớp tích chập (Convolution)
 - Lớp ReLU
 - Lớp tổng hợp (Pooling)
- Lớp làm phẳng (Flatten)
- Lớp đầy đủ cho đầu ra (Output)

Mạng nơ-ron tích chập

- **Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)**
Các mô hình tích chập có ít tham số vì các bộ lọc giống nhau được lặp đi lặp lại, nghĩa là không có quá nhiều trọng số để huấn luyện. Và cuối cùng, phần lớn các mô hình tích chập có thể được tính toán song song, nghĩa là chúng có thể rất nhanh cho mục đích suy luận.





Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Các tầng tích chập
 - Lớp tích chập (Convolution)
 - Tích chập nghĩa là áp dụng một hạt nhân (có dạng một ma trận) vào một ma trận lớn hơn (dữ liệu cần xử lý như hình ảnh) bằng cách trượt nó qua ma trận lớn hơn, tạo thành một ma trận mới.
 - Mỗi phần tử của ma trận mới là tổng của phép nhân từng phần tử của hạt nhân và một phần con của ma trận lớn hơn. Hạt nhân này được áp dụng lặp đi lặp lại khi nó trượt qua ma trận/ảnh.
 - Điều này được thực hiện với số lượng hạt nhân được chỉ định trước để các đặc điểm khác nhau có thể xuất hiện.



Mạng nơ-ron tích chập

➤ **Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)**

- Các tầng tích chập
 - Lớp tích chập (Convolution)

1. Input Image (Grayscale)

0.9	0.1	0.2	0.5	0.8
0.5	0.8	0.1	0.3	0.4
0.2	0.4	0.6	0.1	0.9
0.1	0.5	0.3	0.7	0.2
0.8	0.2	0.4	0.6	0.3

2. Kernel (Weights)

1	0	-1
1	0	-1
1	0	-1

3. Output Feature Map

0.7	0.0	0.0
0.0	0.0	0.0
0.0	0.0	0.0

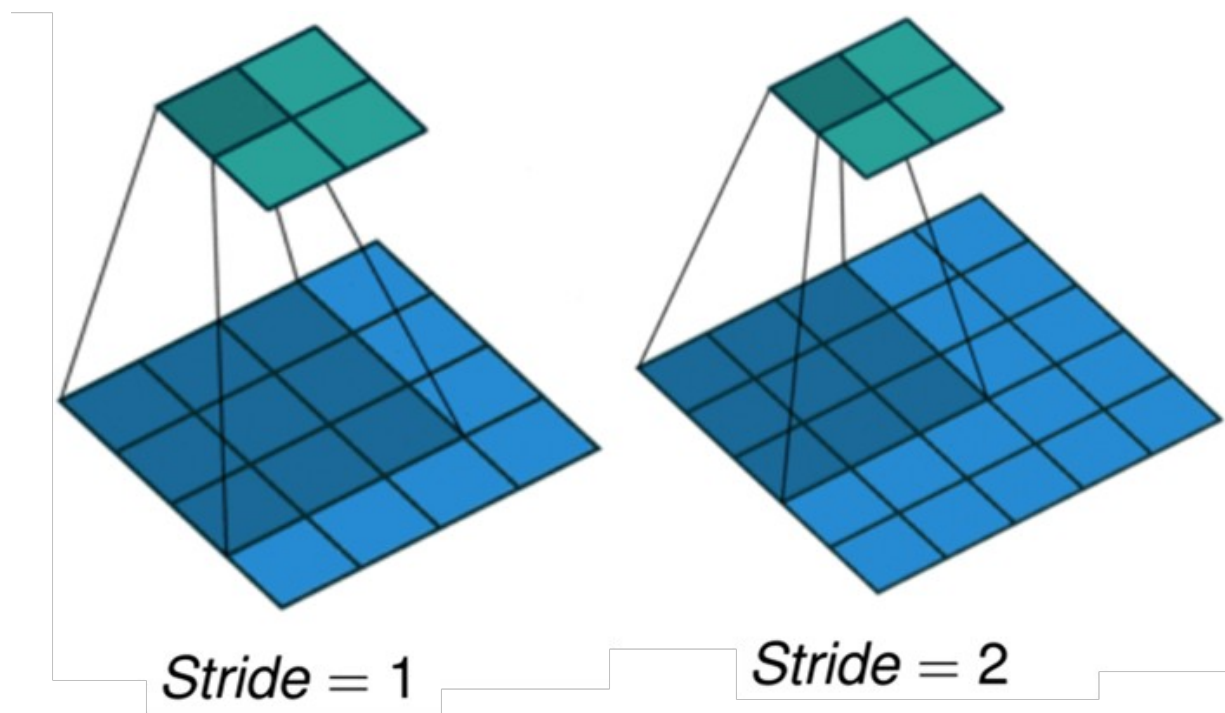
Calculation Detail:

$$\begin{aligned} & (0.9 * 1) + (0.1 * 0) + (0.2 * -1) + \\ & (0.5 * 1) + (0.8 * 0) + (0.1 * -1) + \\ & (0.2 * 1) + (0.4 * 0) + (0.6 * -1) \\ & \quad = 0.70 \end{aligned}$$

Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Các tầng tích chập
 - Lớp tích chập (Convolution)
 - Hạt nhân có thể di chuyển nhiều hơn 1 bước

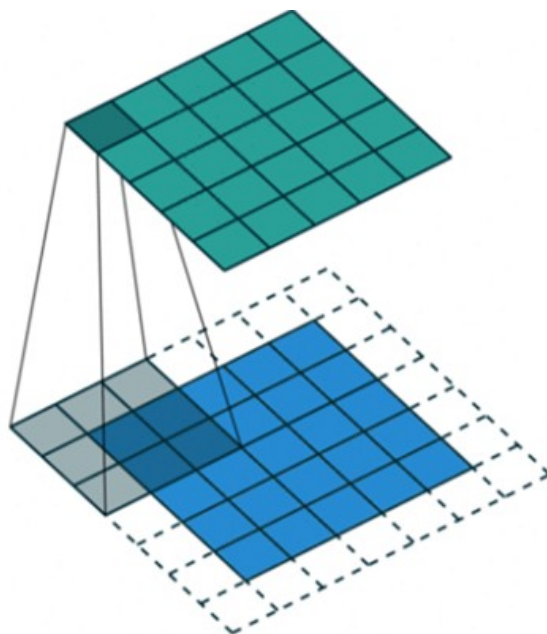




Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Các tầng tích chập
 - Lớp tích chập (Convolution)
 - Trong quá trình di chuyển có thể tại 1 số vị trí bị thiếu dữ liệu cho việc tính tích chập. Khi đó cần thêm (padding).



$$Padding = [1 \ 1]$$

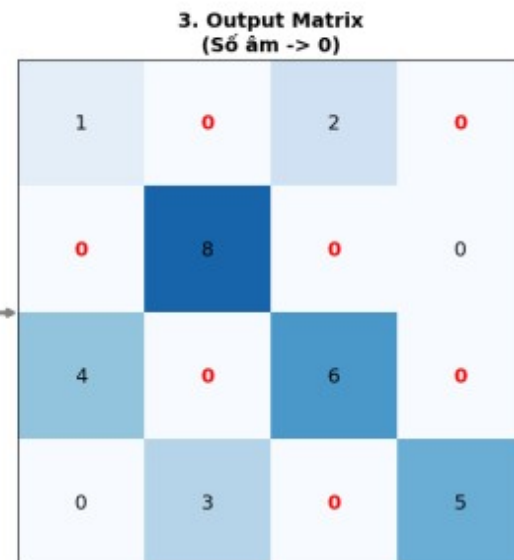
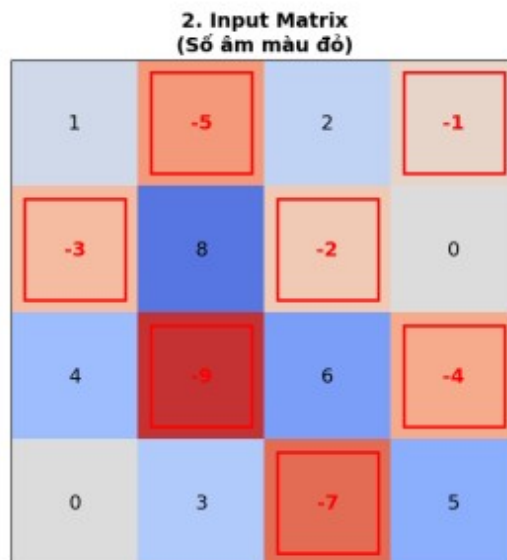
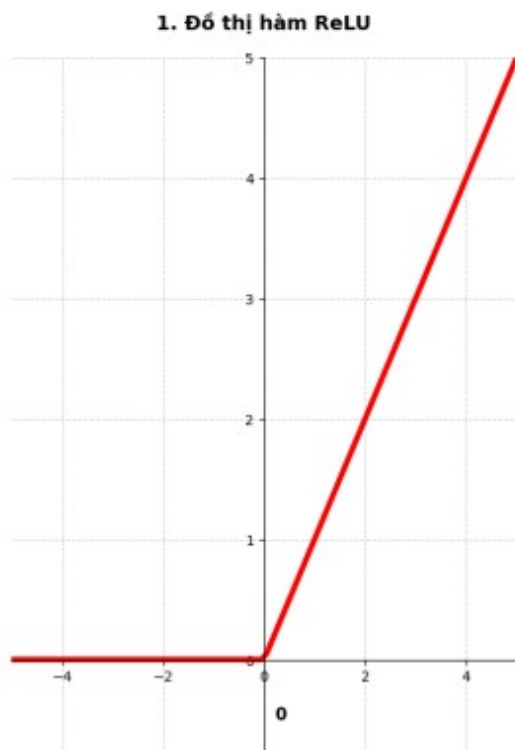


Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Các tầng tích chập

- Lớp ReLU: Kết quả của phép tính tích chập là đầu vào cho hàm kích hoạt. Duyệt qua từng giá trị của khối dữ liệu đầu vào và áp dụng hàm ReLU





Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Các tầng tích chập
 - Lớp tổng hợp (Pooling): Làm giảm kích thước đầu vào. Chia đầu vào thành các vùng có dạng hình chữ nhật và lấy giá trị trung bình hoặc cực đại mỗi vùng.

Input Matrix (4x4)
Chia làm 4 vùng 2x2

12	20	30	0
8	12	2	0
34	70	37	4
112	100	25	12

Output (Max Pooling)
Kích thước giảm còn 2x2

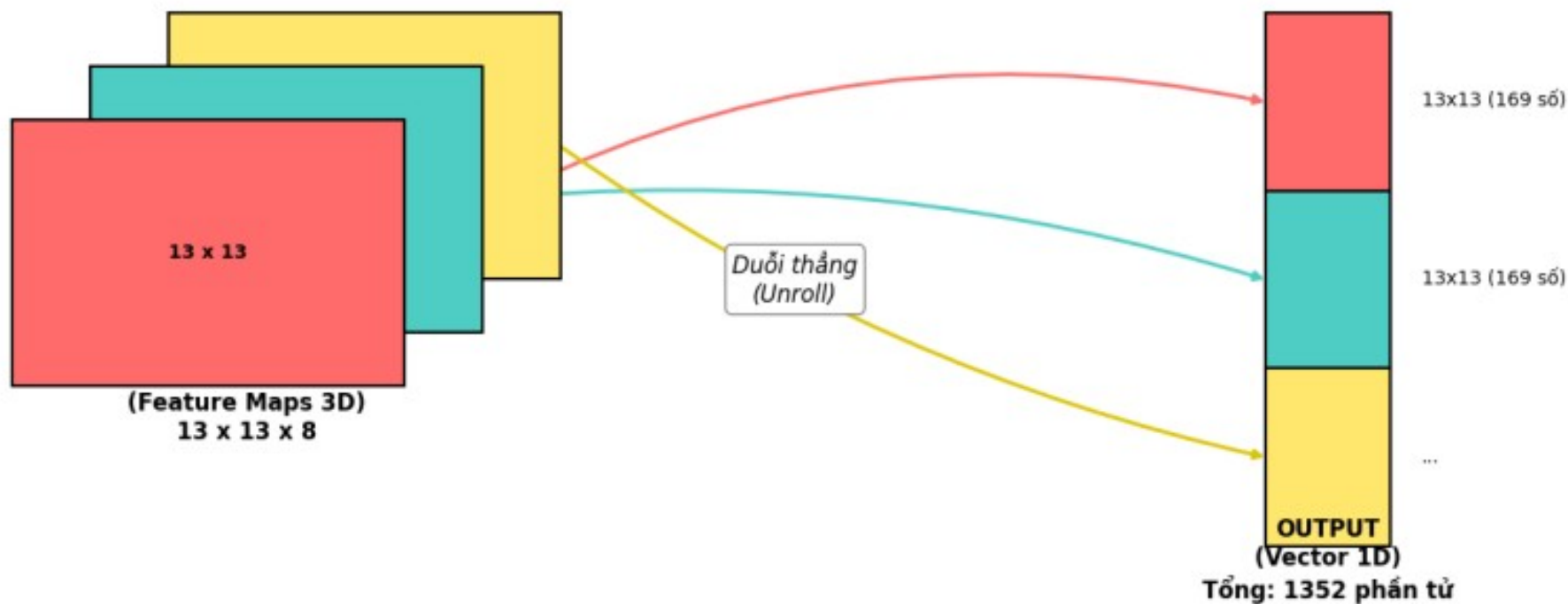
20	30
112	37



Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Lớp làm phẳng (Flatten): làm phẳng dữ liệu đầu ra của tầng tích chập cuối cùng để làm đầu vào mạng đầy đủ phía sau.

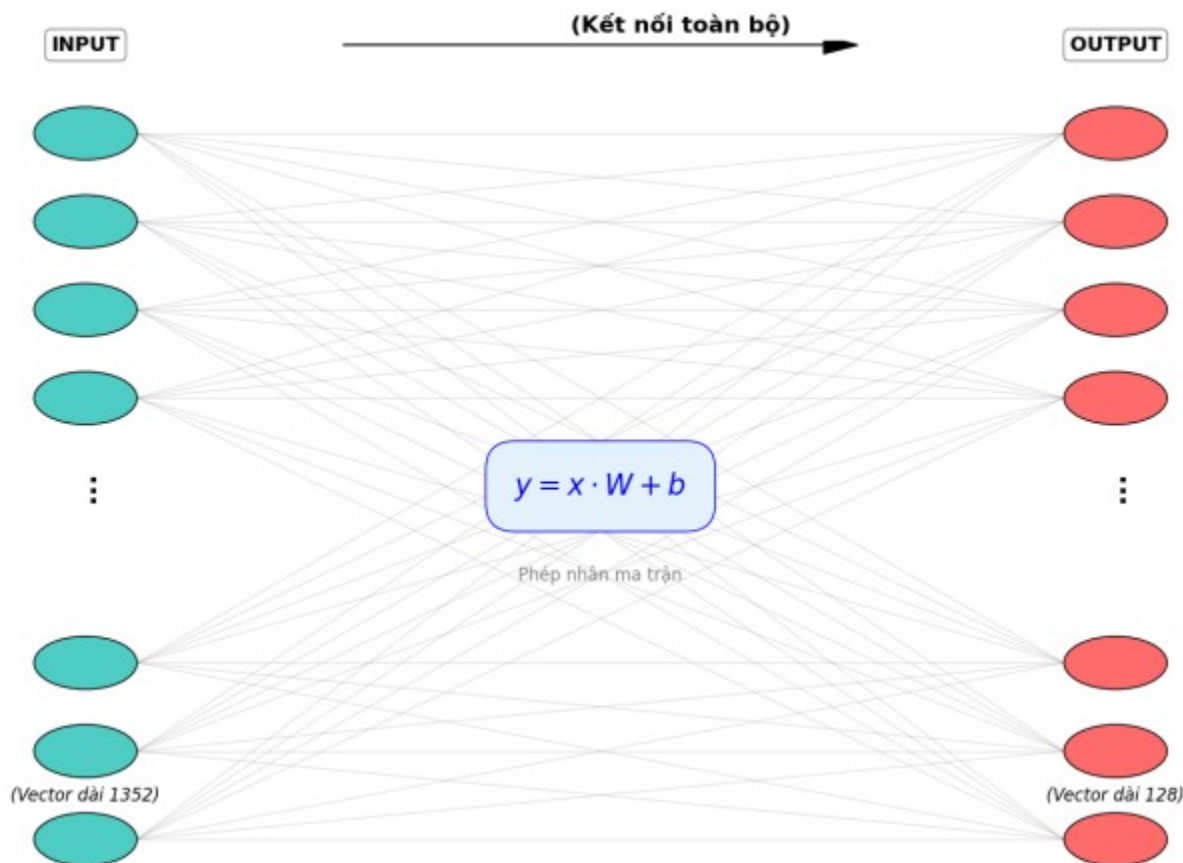




Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Lớp đầy đủ cho đầu ra (Output)





Mạng nơ-ron tích chập

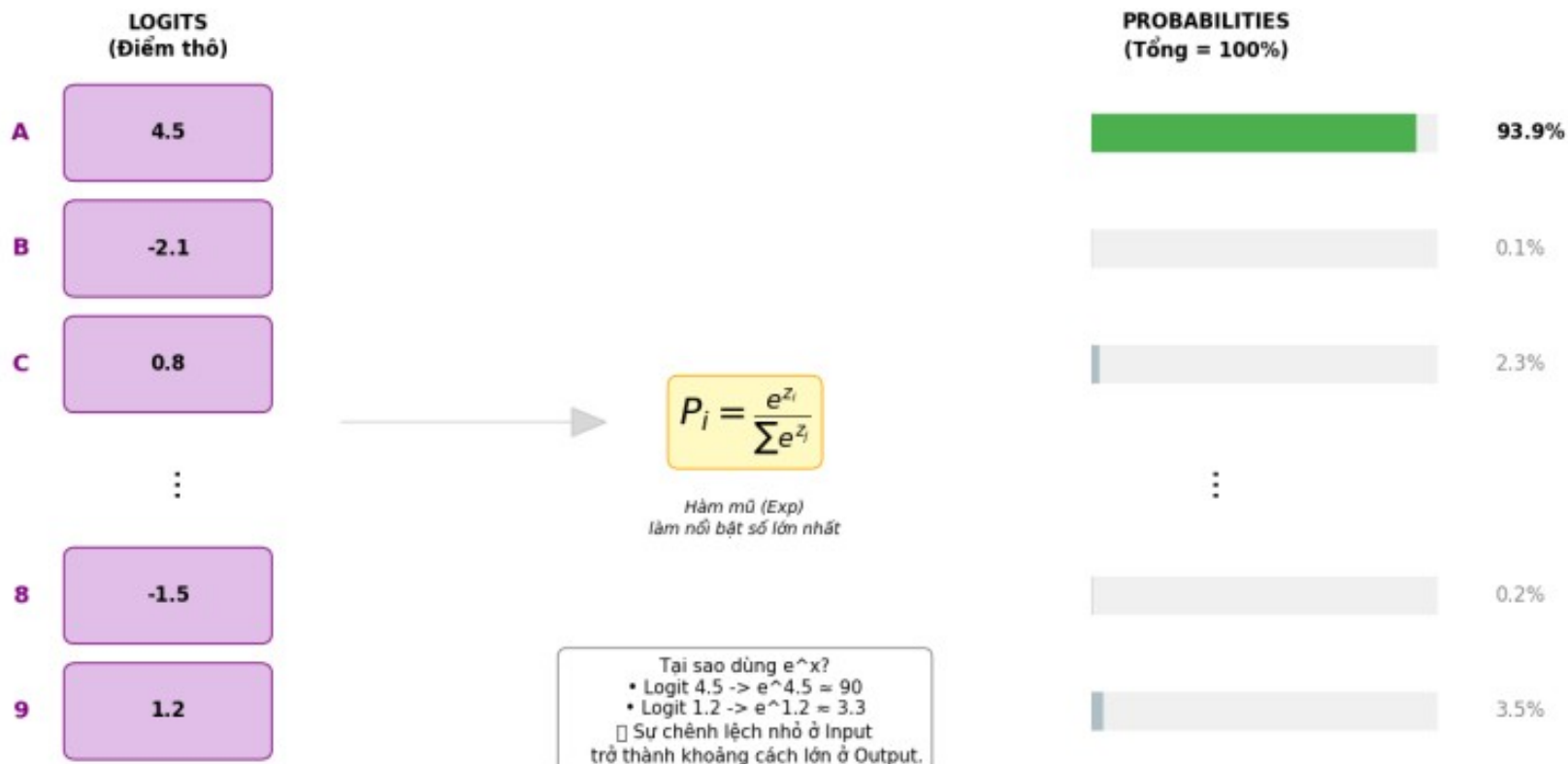
➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Lớp đầy đủ cho đầu ra (Output): đóng vai trò bộ phân loại
 - Hàm softmax còn được gọi là hàm mũ chuẩn hóa, được dùng để tính toán đầu ra của mạng nơ-ron. Hàm này có hai mục đích:
 - Tổng các giá trị đầu ra lúc này sẽ bằng 1, giống như một tập hợp đầy đủ các xác suất.
 - Lưu ý đến hình dạng chung của hàm mũ, các đầu vào lớn hơn có xu hướng trở nên lớn hơn nữa, trong khi các đầu vào nhỏ hơn có xu hướng trở nên tương đối nhỏ hơn. Điều này làm nổi bật các kích hoạt/trọng số mạnh và làm giảm các tín hiệu yếu hơn.

Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Lớp đầy đủ cho đầu ra (Output): đóng vai trò bộ phân loại
 - Hàm softmax

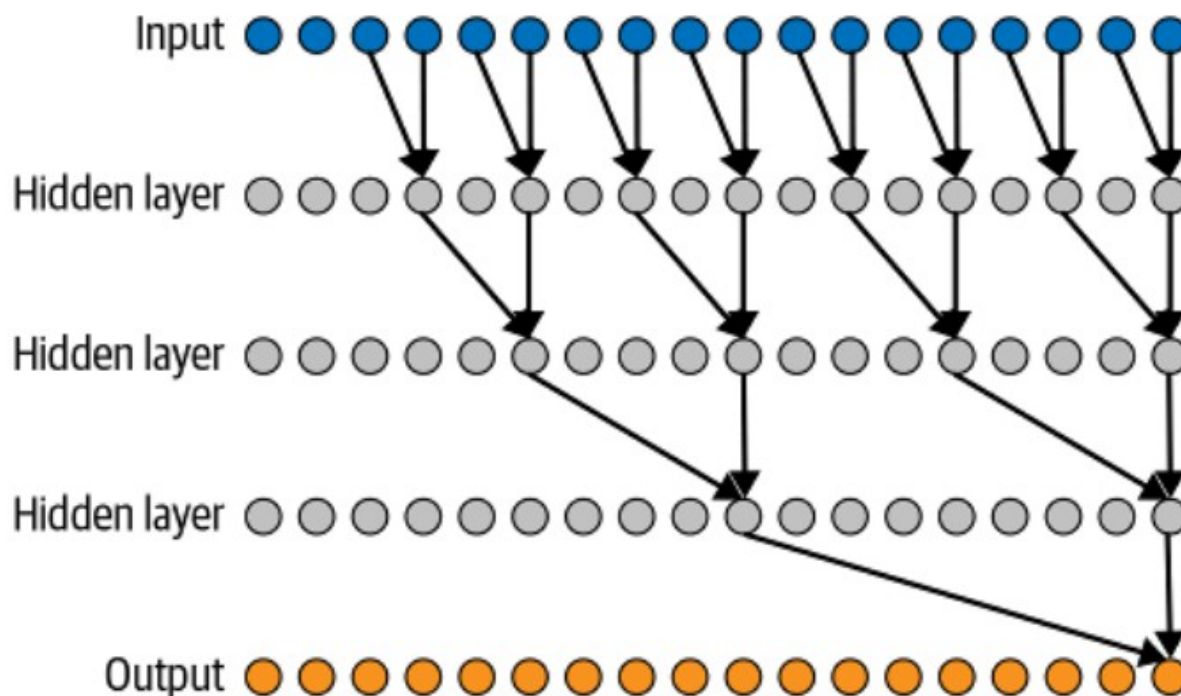




Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- **Tích chập nhân quả** được minh họa thông qua hình ảnh dưới. Tích chập được điều chỉnh để tạo ra tính nhân quả và cảm giác về thời gian.





Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- **Tích chập nhân quả** được điều chỉnh để tạo ra tính nhân quả và cảm giác về thời gian.
 - Phần nhân quả đề cập đến thực tế rằng chỉ những điểm trước đó mới đi vào bất kỳ bộ lọc tích chập nào. Các điểm trước đó sẽ chảy vào các tích chập được sử dụng ở thời điểm sau, nhưng không phải ngược lại.
 - Phần giãn nở đề cập đến việc các điểm bị bỏ qua trong quá trình sắp xếp các bộ lọc tích chập, sao cho bất kỳ điểm nào cũng chỉ đi vào một bộ lọc tích chập ở mỗi cấp độ phân lớp.
 - Điều này thúc đẩy tính thưa thớt của mô hình và giảm thiểu các tích chập dư thừa hoặc chồng chéo, cho phép mô hình nhìn xa hơn về quá khứ trong khi vẫn giữ cho tổng thể tính toán được giới hạn hợp lý.



Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Chuyển đổi chuỗi thời gian thành hình ảnh
 - Bản thân tích chập không hẳn là phù hợp để áp dụng cho chuỗi thời gian đơn biến. Chuỗi thời gian đa kênh có thể phù hợp hơn vì khi đó chúng ta có thể phát triển hình ảnh 2D (hoặc thậm chí 3D) trong đó thời gian chỉ là một trục.
 - Các mô hình tích chập được biết là hoạt động rất tốt trong phân tích hình ảnh, vì vậy, một ý tưởng hay để áp dụng chúng vào chuỗi thời gian là tìm cách chuyển đổi chuỗi thời gian thành hình ảnh. Có nhiều cách để thực hiện việc này, một trong số đó rất thú vị vì nó có thể chuyển đổi ngay cả một chuỗi thời gian đơn biến thành hình ảnh: xây dựng biểu đồ hồi quy.



Mạng nơ-ron tích chập

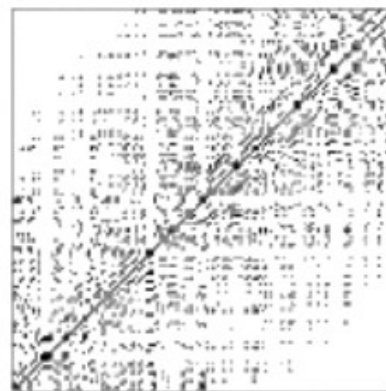
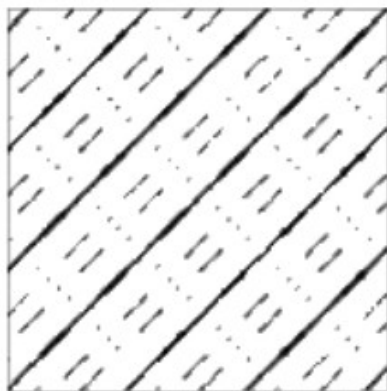
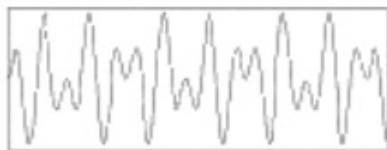
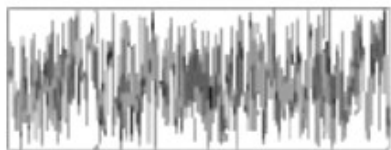
➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Chuyển đổi chuỗi thời gian thành hình ảnh
 - Biểu đồ hồi quy là một cách mô tả, trong không gian pha-trạng thái, khi một chuỗi thời gian trở lại gần đúng pha và trạng thái mà nó đã ở tại một thời điểm trước đó trong thời gian. Điều này được xác định thông qua hàm hồi quy nhị phân.
 - Hồi quy được định nghĩa là $R(i,j) = 1$ nếu $f(i) - f(j)$ đủ nhỏ; nếu không thì bằng 0.
 - Điều này dẫn đến một ảnh nhị phân đen trắng, ví dụ như những ảnh chúng ta thấy trong hình minh họa ở slide sau.
 - Lưu ý rằng i và j đều tham chiếu đến các giá trị thời gian, và trục thời gian không bị giới hạn hoặc bị hạn chế theo bất kỳ cách nào.

Mạng nơ-ron tích chập

➤ Mạng nơ-ron tích chập (Convolutional neural networks - CNNs)

- Chuyển đổi chuỗi thời gian thành hình ảnh
 - Biểu đồ hồi quy



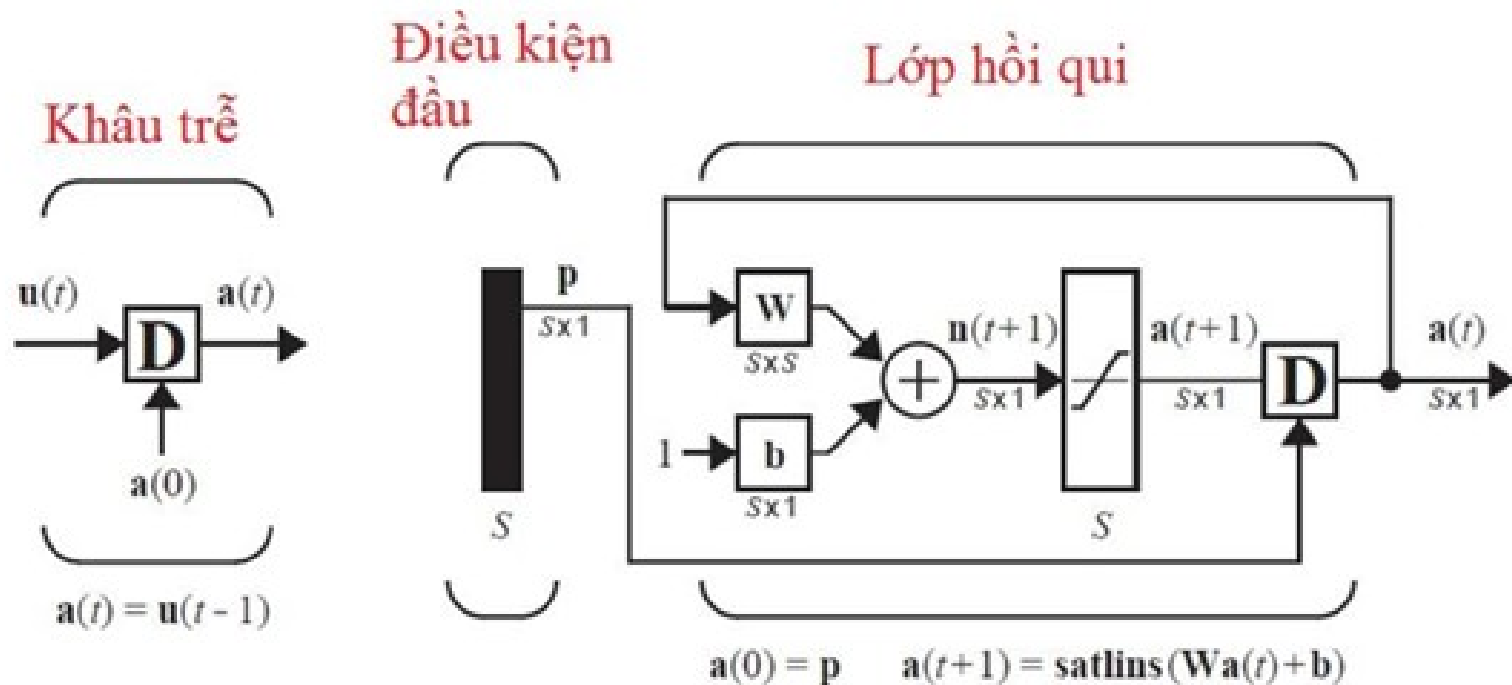


Mạng nơ-ron hồi quy

- **Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)** là một lớp mạng rộng, trong đó các tham số giống nhau được áp dụng lặp đi lặp lại ngay cả khi các đầu vào thay đổi theo thời gian. Những điểm khác biệt quan trọng giữa RNN và mạng truyền thẳng là:
- RNN nhìn thấy từng bước thời gian một, theo thứ tự.
 - RNN có trạng thái mà nó bảo toàn từ bước thời gian này sang bước thời gian khác, và chính trạng thái này cũng như các tham số tĩnh của nó quyết định các cập nhật phản hồi của nó đối với mỗi thông tin mới tại mỗi bước.
 - Do đó, RNN có các tham số giúp nó "cập nhật" trạng thái của mình, bao gồm cả trạng thái ẩn, từ bước thời gian này sang bước thời gian khác.

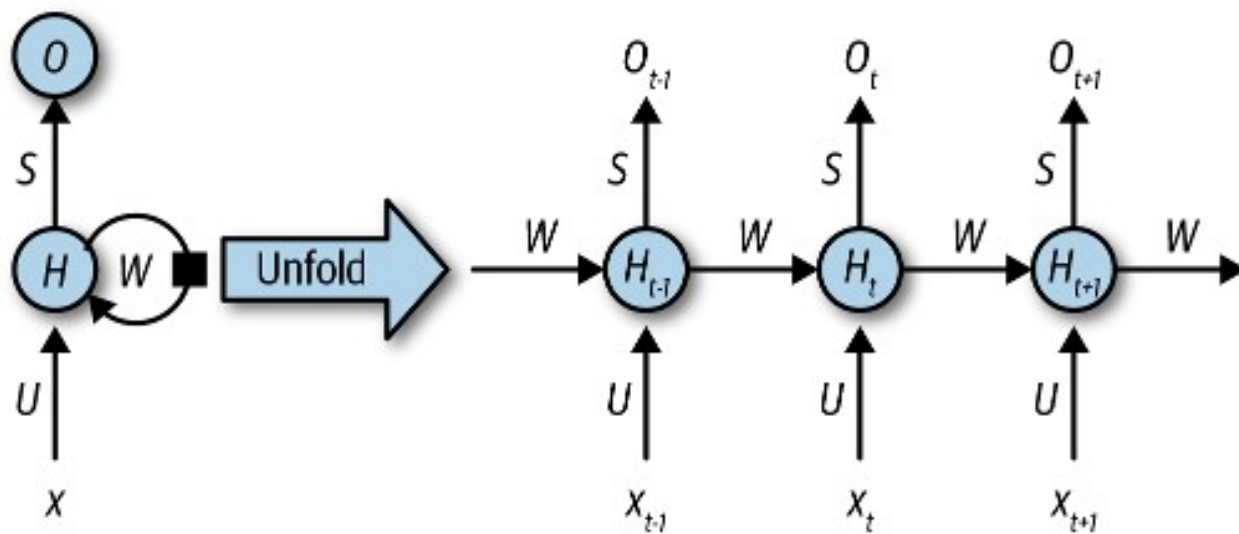
Mạng nơ-ron hồi quy

➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)



Mạng nơ-ron hồi quy

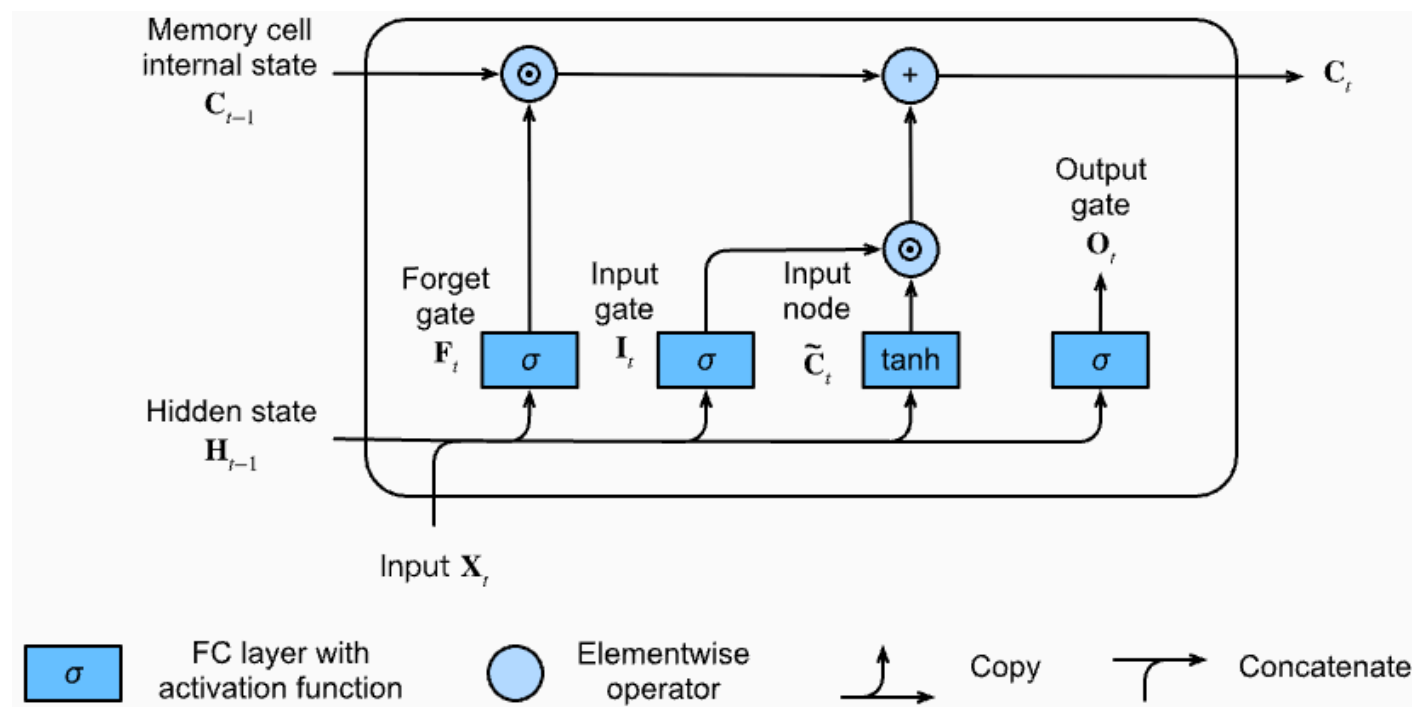
- **Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)** thường được trình bày theo mô hình "mở cuộn" vì kiến trúc hồi quy dựa trên tế bào. Điều này mô tả cách các tham số giống nhau được sử dụng lặp đi lặp lại, do đó số lượng tham số khá nhỏ ngay cả đối với các chuỗi thời gian rất dài.



Mạng nơ-ron hồi quy

➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- **Long Short-Term Memory (LSTM)** được tạo ra để khắc phục việc RNN nhanh quên quá khứ. LSTM thêm một bộ nhớ dài hạn (cell state) và 3 cái “cổng” để điều khiển nó.



Mạng nơ-ron hồi quy

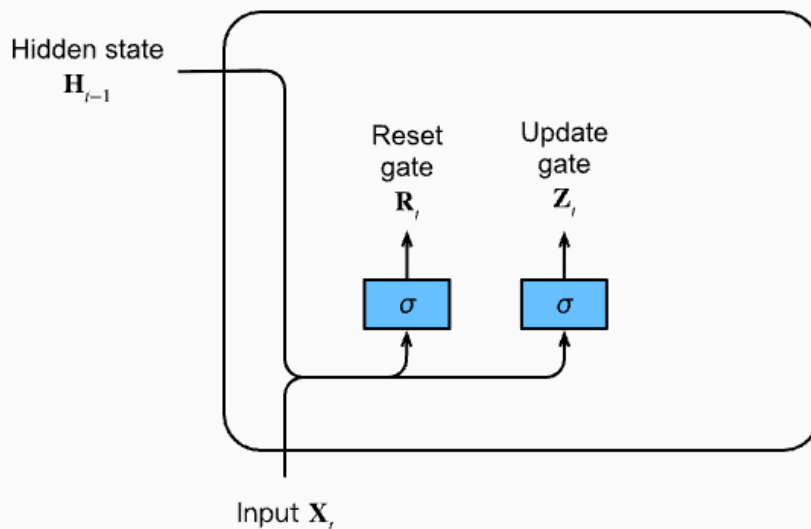
➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- **Long Short-Term Memory (LSTM) có cấu trúc chính như sau:**
 - **Cell state C_t :** “Băng chuyền trí nhớ” chạy xuyên suốt thời gian → giúp không bị quên.
 - **Các cổng điều khiển (gates)**
 - **Forget gate — cổng quên:** Quyết định quên bao nhiêu thông tin quá khứ: $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$
 - **Input gate — cổng ghi nhớ mới:** Quyết định ghi bao nhiêu thông tin mới: $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$
Thông tin mới: $\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$
 - **Output gate — cổng xuất:** Điều khiển trạng thái ẩn h_t
 $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$.

Mạng nơ-ron hồi quy

➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- Đơn vị hồi quy có cổng (Gated Recurrent Unit - GRU) hiện là một trong những phần của RNN được sử dụng rộng rãi nhất. Nó là phiên bản đơn giản hơn của ô Bộ nhớ Dài Ngắn hạn (Long Short-Term Memory - LSTM), hoạt động theo cách tương tự.
 - **GRU** là phiên bản **nhẹ hơn** của LSTM, giảm số cổng từ 3 còn 2.





Mạng nơ-ron hồi quy

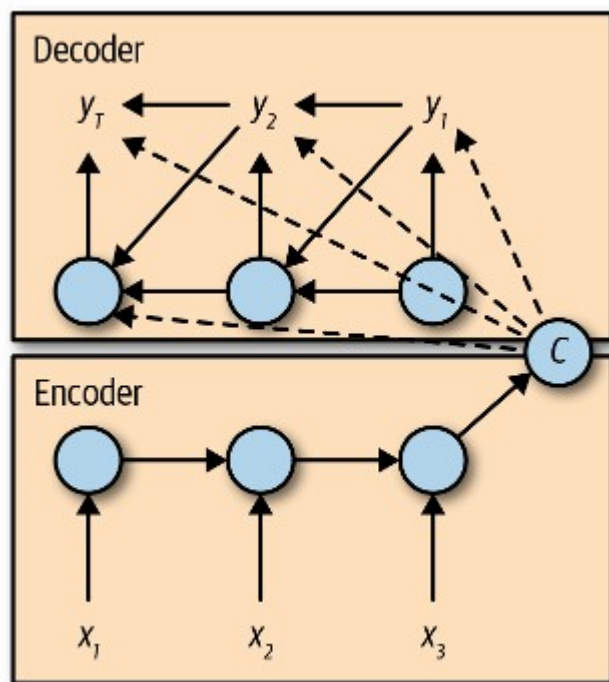
➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- Sự khác biệt giữa GRU và LSTM như sau:
 - GRU có hai "cổng", trong khi LSTM có ba. Các cổng này được sử dụng để xác định lượng thông tin mới được phép đưa vào, lượng thông tin cũ được bảo tồn, v.v. Vì LSTM có nhiều cổng hơn, nên nó có nhiều tham số hơn.
 - LSTM có xu hướng có hiệu suất tốt hơn, nhưng GRU được huấn luyện nhanh hơn (do số lượng tham số). Tuy nhiên, có những kết quả nghiên cứu đã công bố cho thấy GRU vượt trội hơn LSTM. GRU đặc biệt có khả năng vượt trội hơn LSTM đối với các nhiệm vụ phi ngôn ngữ.

Mạng nơ-ron hồi quy

➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- **Autoencoder**, hoặc còn gọi là mô hình seq2seq, đây là một mô hình được sử dụng rất phổ biến và là một phần không thể thiếu trong bộ công cụ học sâu chuỗi thời gian





Mạng nơ-ron hồi quy

➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- **Autoencoder**

- Bộ mã hóa tự động gồm hai lớp tuần hoàn, nhưng không theo nghĩa truyền thống, trong đó mỗi lớp xử lý từng đầu vào một cách tuần tự. Thay vào đó, lớp đầu tiên chạy cho đến khi hoàn tất. Trạng thái ẩn của nó sau đó được chuyển sang lớp thứ hai, lớp này lấy trạng thái ẩn này và đầu ra của chính nó làm đầu vào mới cho bước tiếp theo.
- Mô hình này được xây dựng đặc biệt dựa trên ý tưởng về dịch máy, theo đó việc đưa ra dự đoán tại mỗi bước thời gian là không có ý nghĩa vì trong các ngôn ngữ khác nhau, thứ tự từ và khái niệm có thể rất khác nhau và diễn đạt cùng một điều.



Mạng nơ-ron hồi quy

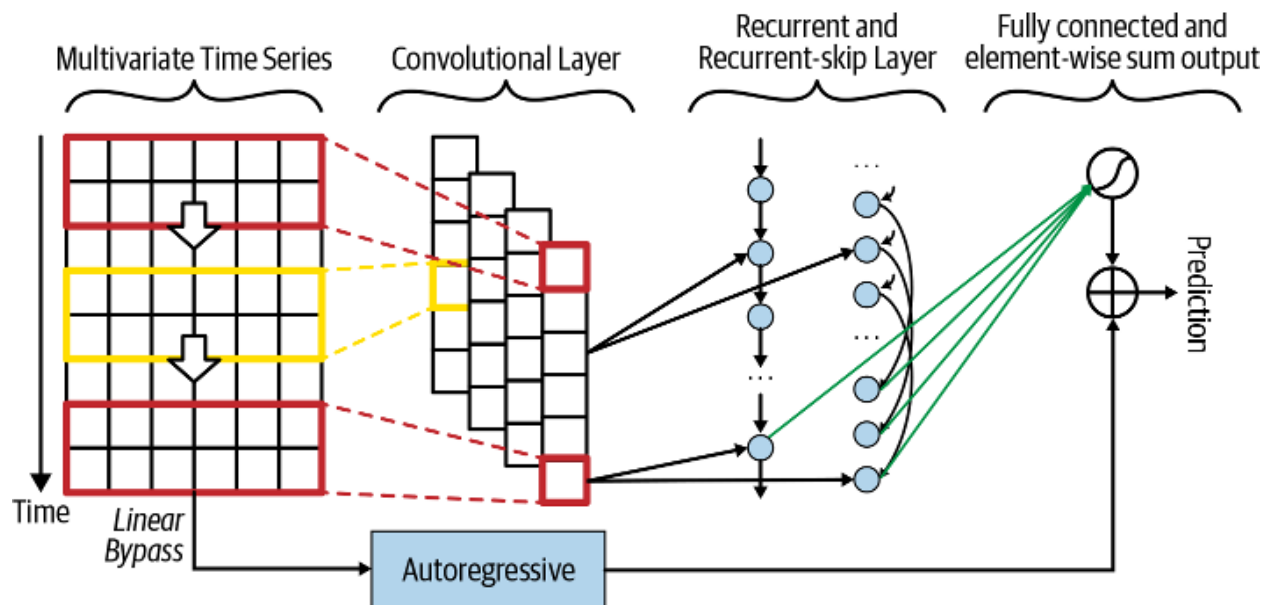
➤ Mạng nơ-ron hồi quy (Recurrent neural networks - RNNs)

- **Autoencoder**

- Ý tưởng là sau khi lớp đầu tiên xử lý hoàn toàn chuỗi thời gian, trạng thái ẩn của nó có thể có một loại chức năng tóm tắt. Bản tóm tắt này sau đó sẽ được đưa vào mô hình mới, mô hình này sẽ dần dần mở rộng ý nghĩa sang ngôn ngữ mới bằng cách kết hợp bản tóm tắt này với đầu ra của chính nó tại mỗi bước thời gian để nó có thể biết được những gì nó đã nói nó cũng hữu ích cho các tác vụ chuỗi thời gian truyền thống hơn, chẳng hạn như dự đoán chuỗi thời gian đơn biến hoặc đa biến.

Kiến trúc kết hợp

- Người ta nhận ra rằng nhiều chuỗi thời gian có xu hướng không được mô hình hóa tốt bởi các mô hình học sâu phi tuyến tính. Điều này có nghĩa là một mô hình học sâu đơn lẻ sẽ không thể hiện được những thay đổi đáng kể về quy mô theo thời gian mà một số tập dữ liệu chuỗi thời gian thể hiện.



Đánh giá

- Vấn đề **đánh giá (evaluation)** trong machine learning và deep learning cho phân tích chuỗi thời gian vô cùng quan trọng, dùng để kiểm tra xem mô hình có bị đánh giá sai hay không:
 - mô hình “ảo tưởng”,
 - không tổng quát
 - và nhất là bị leakage thời gian (một lỗi rất phổ biến).
- Đối với dự báo, việc chọn metric cực kỳ quan trọng. **Các loại sai số (metrics)** trong dự báo time series
 - Sai số cho dự báo giá trị (Forecasting Regression)
 - Metric cho mô hình classification trên chuỗi thời gian

Đánh giá

- Khác với dữ liệu thông thường (tabular), time series có trật tự thời gian và phụ thuộc vào quá khứ. Điều này gây ra 4 vấn đề lớn:
- Không được xáo trộn dữ liệu (no shuffling): Nếu trộn train/test → mô hình nhìn thấy tương lai → đánh giá sai hoàn toàn.
 - Mối quan hệ giữa các điểm không độc lập: Dữ liệu time series có autocorrelation → lỗi không độc lập → metric cần cẩn trọng.
 - Xu hướng (trend) và mùa vụ (seasonality) khiến dự báo ngắn và dự báo dài khác nhau → phải đánh giá theo nhiều horizon.
 - Dữ liệu drift theo thời gian → cần đánh giá theo windows, không phải một lần duy nhất.



Đánh giá

➤ Sai số cho dự báo giá trị (Forecasting Regression)

- MAE – Mean Absolute Error

$$MAE = \frac{1}{n} \sum |y_t - \hat{y}_t|$$

- Dễ hiểu, ít nhạy cảm với outlier.

- RMSE – Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum (y_t - \hat{y}_t)^2}$$

- Phạt nặng các lỗi lớn.

- MAPE – Mean Absolute Percentage Error

$$MAPE = \frac{100}{n} \sum \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

- Dễ theo phần trăm, tuy nhiên, lưu ý tránh khi $y_t = 0$



Đánh giá

➤ Sai số cho phân loại trong chuỗi thời gian

- Một số kí hiệu:
 - TP – True Positive: Mô hình dự đoán Positive, thực tế đúng là Positive
 - TN – True Negative: Mô hình dự đoán Negative, thực tế đúng là Negative
 - FP – False Positive (lỗi loại I, Dự đoán nhầm dương tính): Mô hình dự đoán Positive, thực tế là Negative
 - FN – False Negative (lỗi loại II, Dự đoán nhầm âm tính): Mô hình dự đoán Negative, thực tế là Positive
- Bảng Confusion Matrix

	Thực tế Positive	Thực tế Negative
Dự đoán Positive	TP	FP
Dự đoán Negative	FN	TN

Đánh giá

➤ **Sai số cho phân loại trong chuỗi thời gian**

- Ví dụ TP, TN, FP, FN

Giả sử có 10 mẫu:

○ Thực tế: 1 1 0 0 1 0 1 0 1 0

○ Dự đoán: 1 0 0 0 1 1 1 0 0 0

Ta có:

- TP = 3 (3 trường hợp thực tế 1, dự đoán 1)
- TN = 4 (4 trường hợp thực tế 0, dự đoán 0)
- FP = 1 (thực tế 0 nhưng dự đoán 1)
- FN = 2 (thực tế 1 nhưng dự đoán 0)



Đánh giá

➤ Sai số cho phân loại trong chuỗi thời gian

- Accuracy - Tỷ lệ dự đoán đúng.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision - Trong tất cả các dự đoán là positive, bao nhiêu phần trăm là đúng.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall / Sensitivity / TPR - Trong tất cả positive thật, mô hình tìm được bao nhiêu.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- F1-score - Hòa hợp giữa Precision và Recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$



Đánh giá

➤ Sai số cho phân loại trong chuỗi thời gian

- **Huber Loss** là một hàm loss “lai” giữa **MSE** và **MAE**:

- Gần 0 thì giống **MSE** (nhạy, mượt, dễ tối ưu)
- Xa 0 thì giống **MAE** (ít bị ảnh hưởng bởi outlier)
- **Công thức Huber Loss**

- Cho sai số:

$$r = y - \hat{y}$$

- Huber Loss với tham số δ :

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & \text{nếu } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{nếu } |r| > \delta \end{cases}$$



Đánh giá

➤ Sai số cho phân loại trong chuỗi thời gian

- **Quantile Loss (Pinball Loss)** dùng trong **Quantile Regression**, mục tiêu là dự đoán **một phân vị (quantile)**, ví dụ:
 - 0.1 quantile → mức thấp nhất
 - 0.5 quantile → median
 - 0.9 quantile → mức cao nhất
 - **Công thức Quantile Loss**

Với residual $r = y - \hat{y}$, và quantile $q \in (0,1)$:

$$L_q(r) = \begin{cases} q \cdot r & \text{nếu } r \geq 0 \\ (q - 1) \cdot r & \text{nếu } r < 0 \end{cases}$$